1. Curs 1 - Introducere
2. Curs 2 - Sistemul de fisiere - interfata de user space
3. Curs 3 - Procese
4. Curs 4 - Planificarea executiei. IPC
5. Curs 5 - Gestiunea memoriei
6. Curd 6 - Memoria virtuala
7. Curs 7 - Securitatea memoriei
8. Curs 8 - Fire de executie
9. Curs 9 - Sincronizare
10. Curs 10 - Dispozitive de intrare/iesire

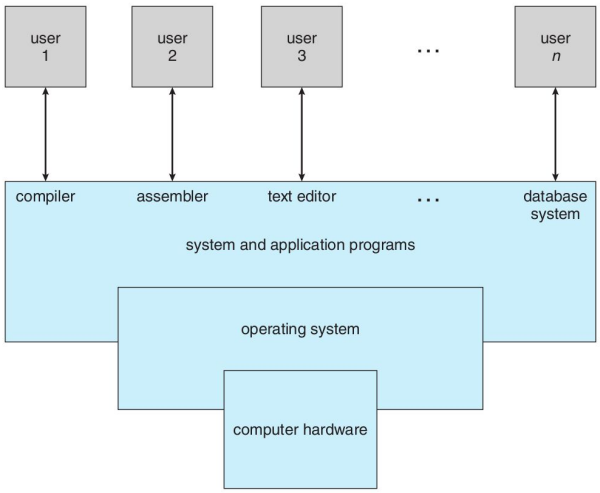
Curs 1 - Introducere

Cuprins
• De ce (un curs) de SO?
• Despre cursul de SO
• Despre sisteme de operare
• Sisteme de calcul. Hardware
• Concepte importante în SO
• Structura unui SO

Sisteme de operare
• Gestiunea resurselor sistemului (hardware)
• Interfaţa dintre aplicaţii şi hardware
• Medierea accesului la hardware
• Securitatea şi integritatea sistemului

De ce sisteme de operare?
• Independent de job/ limbaj de programare/ tip de aplicaţie: Depanare, Performanţă, Securitate
• SO ofera primitive care izoleaza aplicatiile de detaliile hardware
  – POSIX
  – Procese, fisiere, thread-uri, memorie virtuala.

Ce este un SO?



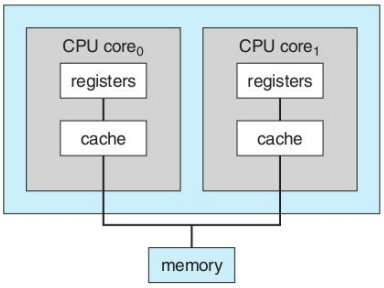
• un set de programe
• vedere top-down: extensie a maşinii fizice
• vedere bokom-up: gestionar al resurselor fizice
• scris în general în C
• relativ transparent utilizatorului (“trebuie să meargă)

Legătura SO - hardware
• SO este primul nivel software peste hardware
• Un SO performant foloseşte facilităţile hardware si expune facilităţile hardware aplicaţiilor
• Hardware-ul are nevoie de SO pentru a putea fi folosit (pentru a construi aplicaţii/software)
• SO evoluează pe măsură ce evoluează hardware-ul

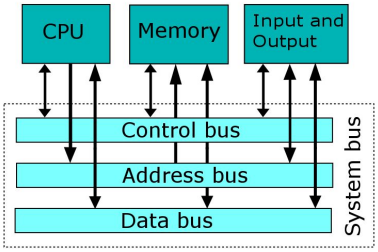
Sistem de calcul
• Sistemul fizic (hardware) + sisteme de operare şi aplicaţii
• Baza pentru a dezvolta aplicaţii, a folosi aplicaţii, a construi sisteme
• În general interconectat cu alte sisteme
• De la servere cu mii de core-uri la dispozitive mici din zona IoT (Internet of Things)

Hardware-ul unui sistem de calcul
• Procesor (CPU) – rulează codul (instrucţiunile) proceselor
• Memorie de lucru (RAM) – stochează datele şi codul proceselor
• Magistrale – leagă CPU, module de memorie, dispozitive de I/E
• Dispozitive periferice (de intrare/ieşire, I/E, I/O) – comunicarea cu exteriorul: utilizator, alte sisteme de calcul, alte dispozitive
• Spaţiu de stocare (disc-uri, flash, ROM, NVRAM)
  – Programe (din care vor lua naştere procese)
  – Date pentru procese
  – Informaţii pentru utilizator (fişiere)

Procesor şi memorie



Magistrale



Memorie

Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	< 16 MB	< 64 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

CONCEPTE IMPORTANTE ÎN SO

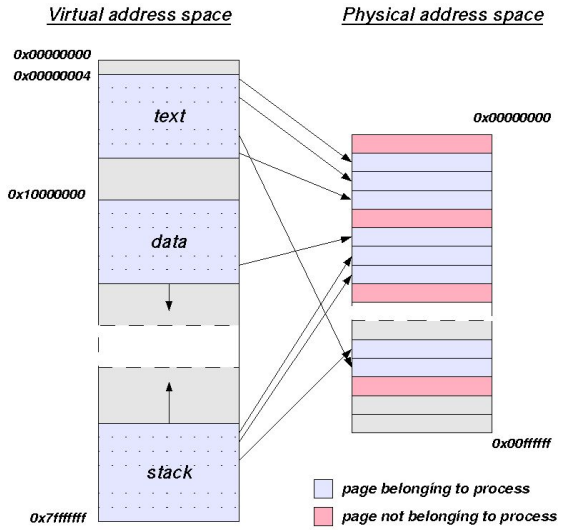
Shell

• Interfaţa utilizator-sistem de operare
• CLI sau GUI
• Bash vs. Windows Explorer / GNOME / KDE
• Un proces care permite pornirea de alte procese/aplicaţii – “shell spawns a process”

Procese

• Iau naştere dintr-un program executabil
• Program în execuţie, entitate dinamică – Noţiunea de “runtime” (timpul rulării) se referă la proces
• Date şi cod în memorie, se execută pe CPU
• Are asociate resurse: spaţiu de adrese de memorie, fişiere deschise, socketi
• SO oferă protecţie şi comunicare inter-proces
• Ierarhie de procese la nivelul SO

Memoria unui proces



Memorie virtuală

• Un proces are un spaţiu virtual (inexistent de fapt) de memorie – Procesul are impresia că toată memoria îi aparţine
• Memoria virtuală decuplează vederea procesului de memoria sistemului
• Procesele lucrează cu adrese virtuale (din spaţiul fizic de adrese) – Adresele fizice sunt adrese din memoria fizică
• SO mapează/asociază spaţiul virtual al proceselor cu memorie fizică

Thread-uri

• Un proces poate avea mai multe thread-uri
• Thread-urile partajează spaţiul virtual de adrese al procesului
• Utile să faci mai multe lucruri cu aceleaşi date (din spaţiul virtual de adrese al procesului)
• Permit folosirea facilităţilor hardware multiprocesor

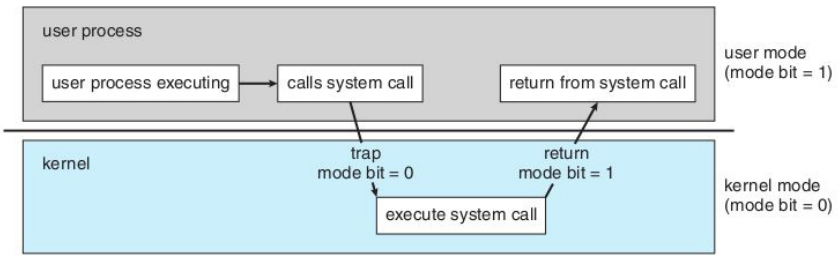
Concurenţă şi sincronizare

• Thread-urile sau procesele pot concura la achiziţia de resurse (date în memorie)
• Accesele concurente pot duce la date inconsecvente sau blocaje
• Sincronizarea garantează accesul consecvent şi ordonat la date

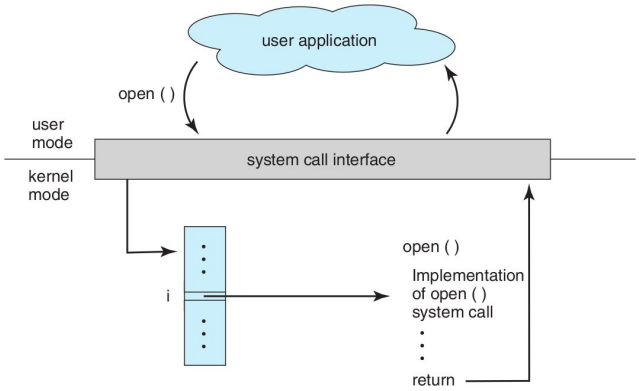
Moduri de execuţie

• Procesorul are un mod privilegiat: supervisor mode, system mode, kernel mode
  – Pentru acţiuni privilegiate
  – Aici rulează sistemul de operare
• Aplicaţiile rulează în modul neprivilegiat: user mode
  – Nu pot comunica cu hardware-ul sau cu alte procese
  – Pot doar acţiona asupra spaţiului de memorie propriu
• Nucleul intermediază accesul proceselor la hardware sau la resursele altor procese
• Separaţia modurilor asigură securitatea sistemului
• Tranziţia user mode -> kernel mode = apel de sistem

Traziţia user mode – kernel mode



Apeluri de sistem

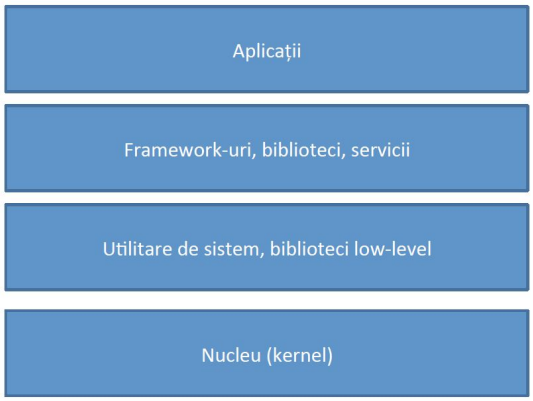


Fişiere

• Unităţi de stocare
• Deschise şi folosite de procese
• Fişier pe disc (static) şi fişier deschis (dinamic, în cadrul unui proces)
• Fişier pe disc: nume, dimensiune, permisiuni, timestamp-uri
• Fişier deschis: handle de fişier, cursor de fişier, drepturi de deschidere, operaţii pe fişier

STRUCTURA UNUI SO

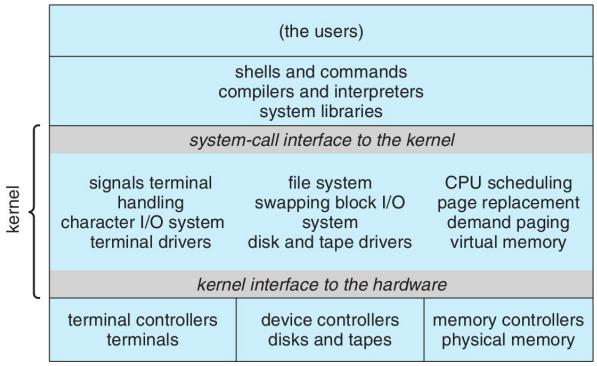
Stiva software pentru un sistem de calcul



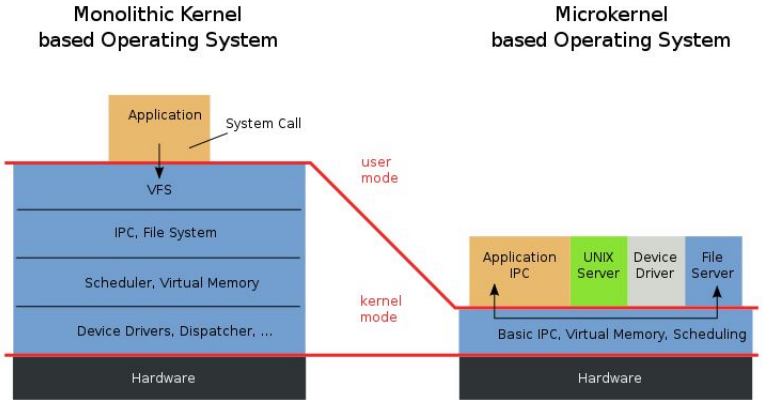
Nucleul (kernel-ul)

• Strict tehnic, nucleul este sistemul de operare
  – Windows are ca nucleul WindowsNT din imaginea ntoskrnl.exe
  – Linux este nucleul unei distribuţii GNU/Linux şi Android
  – Mac OS X şi iOS au ca nucleu XNU
• Încărcat la bootare apoi porneşte primele aplicaţii şi gestionează hardware-ul
• Răspunde apelurilor de sistem ale proceselor
• Gestionează resursele hardware
• Garantează integritatea sistemului

Structura uzuală a SO (monolitic)

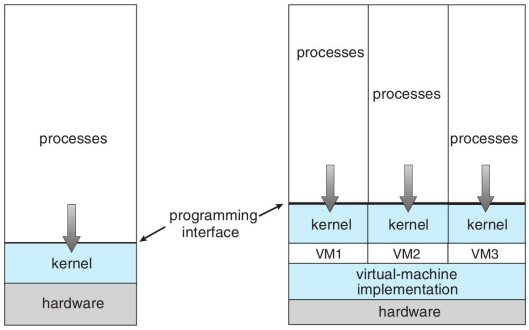


SO monolotic vs. SO microkernel



Monolitic	Microkernel
• Eficient • Coeziunea codului/datelor • Mai puţin flexibil • TCB (Trusted Computing Base) mai mare (design mai puţin sigur)	• Mai lent (comunicare între servicii) • Componentizabil, flexibil, modular • TCB redus (design mai sigur)

Maşini virtuale



Tendinţe curente în SO
• Securitate
• Dispozitive de mici dimensiuni (tinification)
• Scalare (CPU, memorie, disc), maşini virtuale
• Performanţă, suport hardware pentru operaţii specifice

Curs 2 - Sistemul de fisiere - interfata de user space

Cuprins:
- Sisteme de fisiere
- Fisiere. Atribute ale fisierelor
- Operatii cu fisiere
- API pentru lucrul cu fisiere
- Directoare

SISTEMUL DE FISIERE
Ce este un sistem de fisiere?
- Mod de organizare persistenta a informatiei
- Suport pe disc
- Utilizator: ierarhie de fisiere si directoare
- SO: algoritmi si structuri de date
- Performanta, scalabilitate, usurinta in utilizare

De ce sisteme de fisiere?
- Memoria este voltatila, de dimensiune mica
- Suport persistent
- Structurat, organizat

Exemple de sisteme de fisiere: NTFS, ext4, FAT32, ISO9660, HFS+

Operatii cu SF
- Formatare
- Verificarea consistentei
- Montare
- Demontare
- Tuning (optiuni specifice)

FISIERE. ATRIBUTE ALE FISIERELOR
Ce este un fisier?
- Unitatea de baza in sistemul de fisiere
- Abstractizeaza informatia, datele

- Informatia structurata
- Identificat de utilizator prin nume
- Identificat in SF printr-un numar (in Unix - inode number)

Tipuri de fisiere
- Fisiere obisnuite (regular files)
- Directoare
- Link-uri
- Dispozitive de tip caracter
- Dispozitive de tip bloc
- FIFO-uri
- Socketi Unix

Pentru a determina tipul unui fisier in Unix:
- Ls - l
- File
- Stat

Fisiere obisnuite
- De obicei se numesc doar fisiere
- Byte stream - flux de octeti (se citeste si se scrie octet cu octet)
- Organizarea datelor tine de tipul fisierului si de procesul care il foloseste

Directoare
- Colectii de alte fisiere
- Organizate (un vector de alte intrari)

Atribute ale fisierelor (comanda stat)
- Nume, identificator pe disc, dimensiune, drepturi de acces, owner, timpi de acces

Fisiere si fisiere deschise
- Fisierele sunt folosite de procese, iar in momentul folosirii, un fisier este “deschis”
- Un fisier pe disc este static si referit prin nume
- Un fisier deschis este dinamic si referit prin handle

Procese si fisiere
- Fisierele sunt deschise de procese pentru a fi folosite
- Fisierele deschise sunt gestionate prin handle
- Un handle este o referinta la un fisier deschis
- Un proces poate deschide mai multe fisiere
- Un fisier poate fi deschis de mai multe procese (e necesara sincronizarea)
- Un proces are o tabela de handle-uri: sunt retinute referintele la toate fisierele deschise

Cursorul de fisier: file cursor, file pointer, file offset

- Pozitia curenta pentru citire/scriere de un proces
- Incrementat la citire si scriere
- Poate fi repozitionat

Procese, fisiere, handle-uri

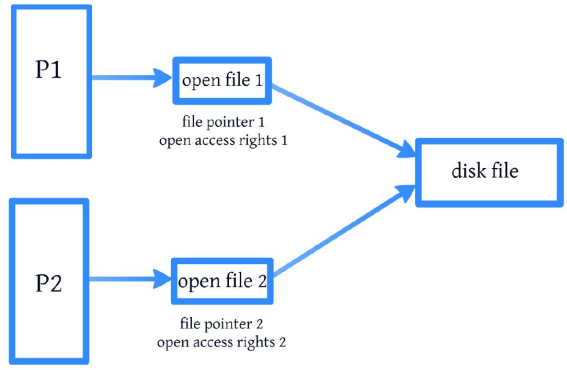
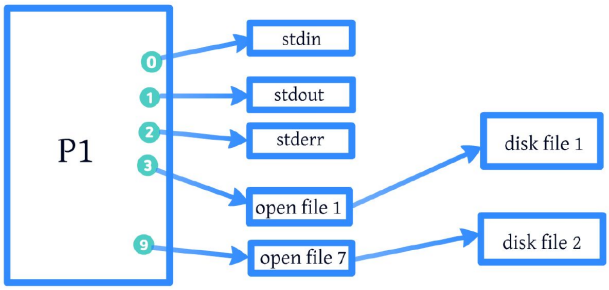


Tabela de descriptori (file descriptor table)

- File descriptor (Unix) = handle



Deschiderea unui fisier (open & close)

- Realizata de un proces
- Intrare: nume de fisier, drepturi de deschidere
- Iesire: handle de fisier
- Se aloca o structura de fisier deschis
- La inchiderea fisierului, se invalideaza handle-ul, se dezaloca structura de fisier deschis

Citire si scriere (read & write)

- Pe un fisier deschis, pe un handle
- La intrare: handle, cat se scrie/citeste, unde/de unde
- La iesire: cat s-a scris/citit
- Dupa citire, scriere, se avanseaza cursorul de fisier

Pozitionare (seek)

- Se comanda plasarea cursorului de fisier
- Poate fi dat inapoi (rewind)
- Input: handle, cu cat se muta, in raport cu ce
- Output: noua pozitie

API DE LUCRU CU FISIERE

Shell

- Touch file.txt
- Cat file.txt
- Echo “abc” > file.txt
- Rm file.txt

ANSI C

- FILE *f = fopen(“file.txt”, “r+”)
- N = fread(f, sizeof(char), 100, buffer)
- M = fwrite(f, sizeof(char), 200, buffer2)
- fseek(f, 100, SEEK_SET)
- fclose(f)

Unix(POSIX)

- Int fd = open(“file.txt”, O_RDWR)
- N = read(fd, buffer, 100)
- M = write(f, buffer2, 200)
- lseek(fd, 100, SEEK_SET)
- close(fd)

Windows(Win32)

- HANDLE f = CreateFile(“file.txt”, GENERIC_WRITE, …, OPEN_EXISTING, ...)
- Ret = ReadFile(f, buffer, 100, &bytesRead, NULL)
- SetFilePointer(f, 100, NULL, FILE_BEGIN)
- CloseHandle(f)

Java

- FileReader fr = new FileReader(“f.txt”)
- fr.read(buffer)
- fr.close()
- FileWriter fw = new FileWriter(“f.txt”)
- fw.write(buffer)
- fw.close()
- FileChannel fc = FileChannel.open(“f.txt”, READ, WRITE)
- fc.position(100)
- Fc.close

Python

- F = open(“f.txt”, “r+”)
- Str = f.read(10)
- f.write(buffer)
- f.seek(0,0) #place at beginning
- f.close()

REDIRECTARI

Ce inseamna redirectare?

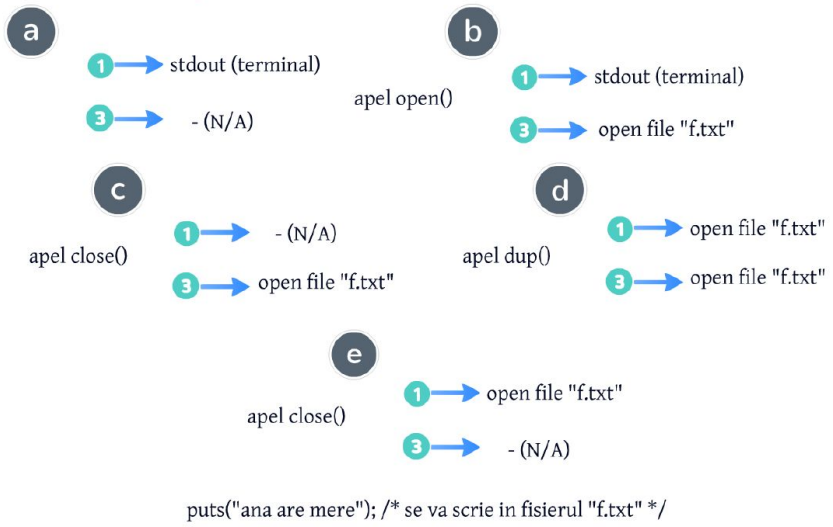
- Iesirea standard este transmisa intr-un fisier: echo “ana” > file.txt
- intrarea standard este transmisa dintr-un fisier: grep “ana” < /etc/passwd
- Este transmisa intr-un/dintr-un pipe: grep “ana” /etc/passwd | wc -l

De ce redirectare?

- Pentru a retine iesirea unei comenzi
- Pentru a transmite intrarea unei comenzi dintr-un fisier
- Pentru comunicare inter-proces(pipe), filtre
- Pentru a anula iesirea unei comenzi (/dev/null)

Implementare redirectare (echo “ana > f.txt”)

- Fd = open(“f.txt”, O_WRONLY)
- Close(STDOUT_FILENO)
- dup(fd)
- close(fd)
- Dup duplica un descriptor in primul descriptor liber, iar primul descriptor liber de mai sus e STDOUT_FILENO



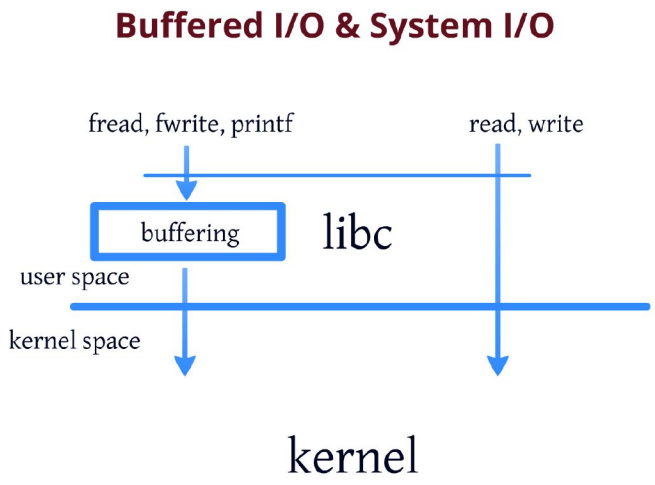
BUFFERED I/O vs SYSTEM I/O

Buffered I/O (ANSI C I/O)

- Biblioteca standard C face buffering la informatii
- Nu sunt transmise instant catre sistemul de operare
- printf, fprintf, puts, fputs, fwrite
- Sunt transmise sistemului de operare: la fflush(), la fclose(), line buffered (standard input), fully buffered(fisiere): cand s-a umplut bufferul

System I/O (low-level I/O)

- Datele sunt transmise de la/catre SO
- Nu exista buffering
- Apelul se intoarce cand o parte din date au fost transmise
- Read, write



Buffered I/O	System I/O
- Buffered - Latenta in propagare - Memorie ocupata - Mai putine apeluri de sistem	- Sincron - Transmitere directa - Doar bufferele programului - Overhead de apel de sistem

Curs 3 - Procese

Cuprins
• Rolul proceselor
• Atributele unui proces
• Planificarea proceselor
• Crearea unui proces
• Alte operaţii cu procese
• API pentru lucrul cu procese

Ce este un proces?
• Modul în care se execută acţiuni în sistemul de operare
• Un program aflat în execuţie
• Încapsularea/abstractizarea execuţiei în SO
• Abstractizare peste procesor

De ce procese?
• Ca să putem efectua acţiuni în SO
• Ca să avem o separare a acţiunilor: un proces face ceva, altul face altceva
• Ca să avem izolare: un proces nu încurcă alt proces
• Pentru modularizare: când ceva pică sau nu merge, să putem determina rapid problema

ROLUL PROCESELOR
Ce face un proces?
• Rulează instrucţiuni pe procesor – Instrucţiunile sunt în memoria procesului (cod)
• Lucrează cu informaţii de la input/output – Fişiere, reţea, tastatură, monitor
• Interacţionează cu alte procese – Comunicare, partajare resurse

Ce conţine un proces?
• Memorie: cod, date
• Tabelă de descriptori de fişiere
• Legături către alte procese (proces părinte, procese copil)

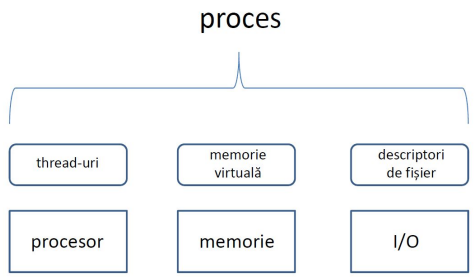
Proces şi procesor
• Un proces rulează instrucţiuni pe procesor
• Procesele sistemului au nevoie de procesoare
• De obicei procesele sistemului sunt mai multe decât procesoarele sistemului – E nevoie de planificarea proceselor pe procesoare

Proces şi memorie
• Un proces are o memorie proprie (izolată de alte procese)
• Cod/instrucţiuni şi date

• Instrucţiunile sunt aduse din memoria RAM în procesor şi executate – Spunem că procesul se execută pe procesor

Proces şi I/O
• Un proces comunică cu exteriorul: disc, reţea, tastatură, monitor
• Comunicarea se face, de regulă, prin decriptori de fişier
• Un proces are o tabelă de descriptori de fişier
• Un descriptor de fişier referă un fişier, socket, terminal, dispozitiv etc.
• Operaţiile cu I/O blochează procesul – I/O este mai lent decât procesorul – procesul aşteaptă încheierea operaţiei

Procesor, memorie, I/O

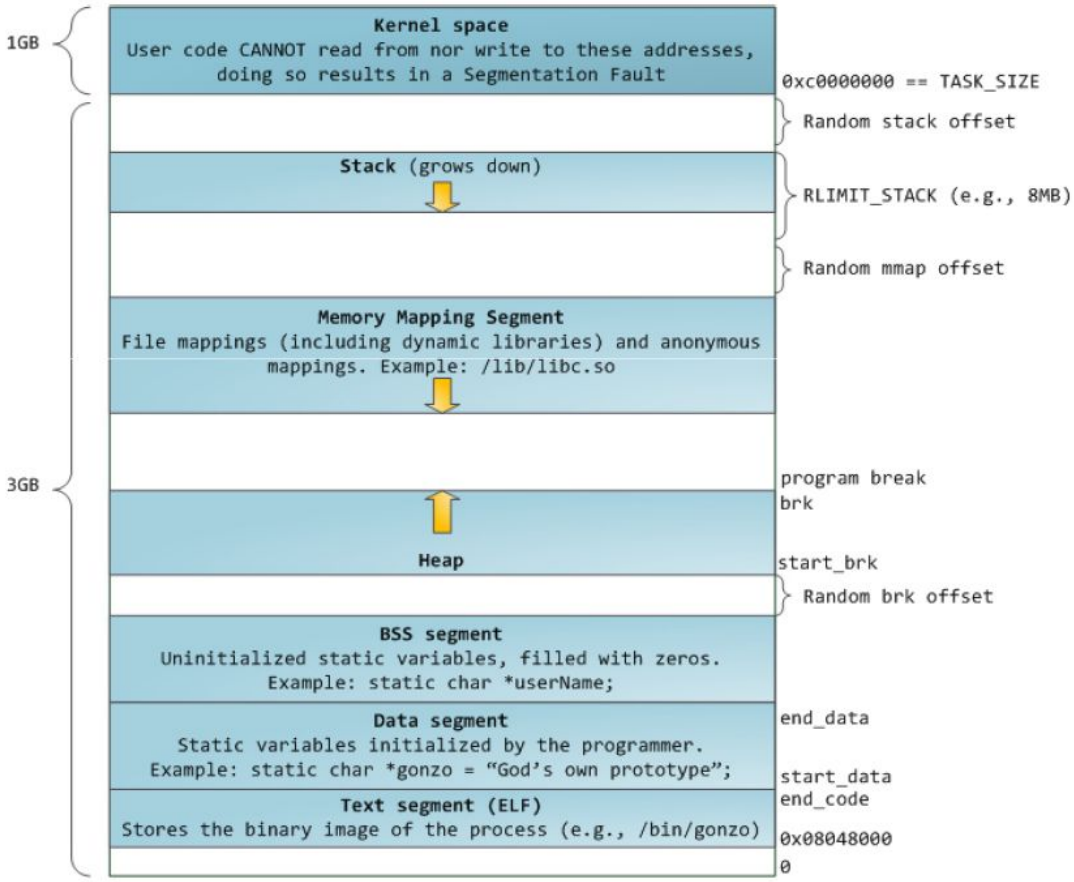


Tipuri de procese
• CPU bound (CPU intensive) – rulează des pe procesor
• I/O bound (I/O intensive)
    – rulează rar pe procesor
    – fac operaţii de I/O -> se blochează
• Interactive (foreground) – interacţionează cu utilizatorul
• Neinteractive (batch, background) – servicii, daemoni

ATRIBUTELE UNUI PROCES
Cum arată un proces la nivelul SO?
• O structură de date
    – PCB (Process Control Block)
    – Descrie un proces la nivelul SO
• Informaţii legate de resursele folosite
• Un identificator (PID: process identifier)
• Legături cu celelalte structuri
• Informaţii de securitate, monitorizare, contabilizare

Resursele unui proces
• Timp de rulare pe procesor
• Memorie (cod şi date)
• Tabelă de descriptori de fişier
• Unele resurse pot fi partajate cu alte procese

Spaţiul (virtual) de adrese



• Fiecare proces are un spaţiu (virtual) de adrese
• Asigură izolarea faţă de alte procese
• Procesul are impresia folosirii exclusive a memoriei
• Toate resursele de memorie (cod, date, biblioteci, stivă) sunt referite prin zone în spaţiul virtual de adrese

Tabela de descriptori
• Proprie fiecărui proces
• Interfaţa pentru I/O a unui proces
• Vectori de pointeri către structuri de fişiere deschise
• Structurile pot referi fişiere, socketi, dispozitive speciale (terminale)

Atribute ale unui proces
• PID
• parent PID
• pointeri către resurse
• stare (de rulare, aşteptare)

• cuantă de timp de rulare
• contabilizare resurse consumate
• utilizator, grup

PLANIFICAREA PROCESELOR

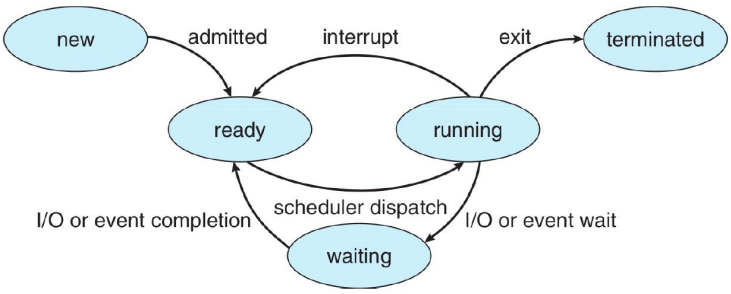
Procese şi procesoare
• Pentru a rula un proces are nevoie de un procesor
• Procesorul rulează instrucţiunile procesului
• Procesele într-un sistem sunt mai multe ca procesoarele
• Sistemul de operare asigură accesul echilibrat al proceselor la procesoare

Multitasking
• SO schimbă rapid procesele pe procesoare
• După un timp (cuantă, time slice) un proces este scos de pe procesor şi pus altul în loc – se spune că "expiră cuanta" – acţiunea este numită "schimbare de context" (context switch)
• Cuantă de timp de ordinul milisecundelor – se schimbă foarte rapid procesele – impresia de rulare simultan
• Un proces poate fi întrerupt în momentul executării unei secvenţe

Starea unui proces
• rulare (RUNNING) – procesul este pe un procesor şi rulează
• aşteptare (WAITING) – procesul a executat o acţiune blocantă (de exemplu citire I/O) şi aşteaptă sosirea datelor; nu poate rula
• gata de execuţie (READY) – procesul poate să ruleze pe procesor – pe moment nu are loc pe procesor, e alt proces acolo

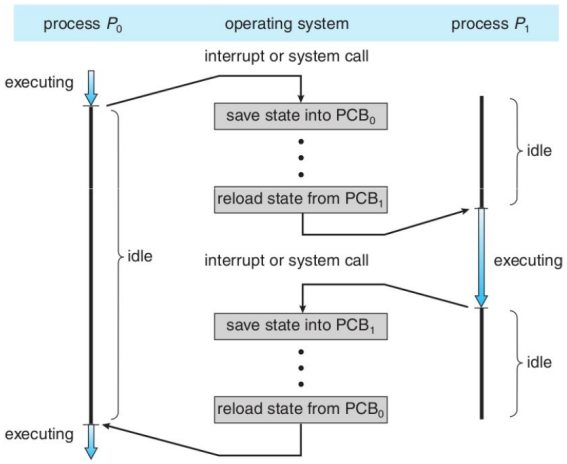
Planificarea unui proces (scheduling)
• Un proces este adus din starea READY în starea RUNNING
• Este adus dacă există un procesor liber
• Un procesor este liber dacă procesul de pe procesor l-a eliberat
• Se spune că are loc o schimbare de context (context switch)

Tranziţii între stările unui proces



• RUNNING -> READY
  – procesului i-a expirat cuanta
  – există un alt proces în starea READY cu prioritate superioară
• RUNNING -> WAITING
  – procesul a executat o operaţie blocantă
• WAITING -> READY – evenimentul aşteptat de proces s-a produs
• READY -> RUNNING
  – s-a eliberat un procesor
  – procesul este primul din coada de procese READY

Schimbarea de context



• Un proces este înlocuit pe procesor cu alt proces
• Se salvează procesul/contextul vechi
• Se restaurează procesul/contextul nou
• O schimbare de context înseamnă overhead
  – mai multe schimbări de context: mai mult overhead
  – mai puţine schimbări de context: mai puţină interactivitate

Planificatorul de procese
• Componentă a SO
• Responsabilă cu planificarea proceselor
  – asigurarea accesului proceselor la procesoare
  – compromis (trade-off) între echitate şi productivitate

CREAREA UNUI PROCES
Cum ia naştere un proces?
• Din cadrul unui executabil (program)
• Un alt proces (părinte) creează un proces (copil)
  – noul proces (procesul copil) îşi populează memoria cu informaţii din executabil
• Acţiunea se mai cheamă loading, load time – realizată de loader

Ierarhia de procese

• Un proces poate crea unul sau mai multe procese copil
• Un proces poate avea un singur proces părinte
• În Unix, procesul init este în vârful ierarhiei de procese
  – init este creat de sistemul de operare la boot
  – init creează apoi procesele de startup
  – procesele de startup creează alte procese etc

Procese şi executabile

• Unul sau mai multe procese iau naştere dintr-un fişier executabil
• Fişierul executabil conţine în mod esenţial datele şi codul viitorului proces
• Procesul are zone de memorie care nu sunt descrise în executabilul aferent
  – stiva
  – heap-ul (malloc)
  – zone pentru bibliotecile dinamice

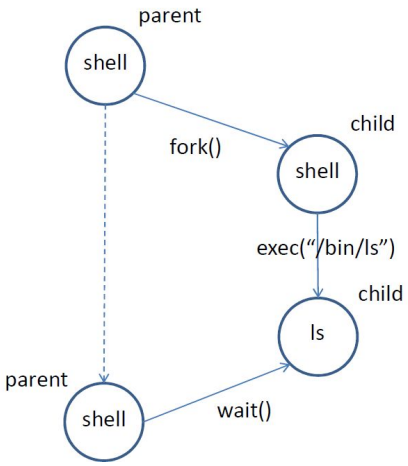
Funcţionarea unui shell

• Se scrie la stdin un şir
• Şirul este interpretat de shell într-o cale de executabil (şi argumente)
• Procesul shell creează un nou proces (copil)
• Procesul copil "încarcă" (load) datele şi codul din executabil
• Procesul copil este planificat de SO
• Părintele procesului copil este procesul shell

fork şi exec

• Separare între crearea unui nou proces şi încărcarea datelor dintr-un executabil
• fork: creare nou proces (copil) (aproape identic procesului părinte)
• exec: încărcarea informaţiilor dintr-un executabil în memoria procesul copil

fork, exec şi shell-ul



fork, exec şi redirectare

```
pid = fork();
if (pid == 0) { /* child process */
        fd = open("a.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
        dup2(fd, STDOUT_FILENO);
         execl("/bin/ls", "/bin/ls", NULL);
}
```

ALTE OPERAŢII CU PROCESE

Încheierea execuţiei unui proces

• Procesul îşi încheie execuţia
  – a ajuns la sfârşitul programului
  – a apelat exit()
• Decizia este a procesului
• În final, procesul nu mai există în sistem

Terminarea unui proces

• Sistemul de operare încheie procesul
  – la cererea altui proces (kill)
  – procesul a efectuat o operaţie nevalidă (segmentation fault)
• Se trimite un semnal (Unix) sau o excepţie (Windows)
• În final, procesul nu mai există în sistem

Aşteptarea încheierii unui proces

• Sincronizarea acţiunii unor procese
  – faci acţiunea X după acţiunea Y
  – procesul care face acţiunea X aşteaptă încheierea execuţiei procesului care face acţiunea Y
• Se cheamă "waiting for a process"
• Sunt utile informaţiile legate de încheierea procesului
  – valoarea de retur, în shell variabila $?

Operatorul shell &

• În mod obişnuit shell-ul aşteaptă încheierea proceselor create
• Operatorul &: shell-ul nu mai aşteaptă încheierea procesului
  – procesul rulează în background
  – shell-ul controlează stdin

Proces orfan

• Un proces al cărui părinte şi-a terminat execuţia
• Îşi pierde legătura în ierarhia de procese
• Este adoptat de un proces dedicat (init pe Unix)

Proces zombie
• Un proces care s-a încheiat dar nu a fost aşteptat
• Rămân informaţii reziduale care vor putea fi folosite de procesul părinte
• Dacă un proces zombie rămâne şi orfan, este adoptat de init şi apoi este încheiat
• Procesele zombie pe durată mai lungă ocupă (degeaba) resurse ale sistemului

Comunicare interprocese
• Transfer de date între procese
• Partajare de date (de obicei memorie) între procese
• Comunicarea prin fişiere este modul cel mai simplu (şi barbar)
• De avut în vedere sincronizarea proceselor
  – un proces citeşte dacă altul a scris
  – un proces poate scrie dacă altul e pregătit să citească
  – de obicei se folosesc buffere (zone de memorie intermediare)

pipe-uri
• Pipe-uri anonime (spre deosebire de FIFO-uri, pipe-uri cu nume)
• Buffere de comunicare între procese
• Expuse ca o “ţeavă” între două procese
  – un capăt de scriere
  – un capăt de citire
  – cele două capete sunt descriptori de fişier
• Se folosesc operaţiile de tip read şi write
• Procesele trebuie să fie “înrudite”

API DE LUCRU CU PROCESE
Lucrul cu procese în shell
• Se creează prin rularea de comenzi
• Se încheie la încheierea rulării comenzii
• Se termină folosind kill, pkill, Ctrl+c, Ctrl+\
• Shell-ul este procesul părinte al proceselor nou create

Lucrul cu procese în ANSI C
```
system("ps -u student");
FILE *f = popen("ps -u student", "r");
pclose(f);
```
• nu e ANSI, e POSIX, dar are suport si pe Windows

Lucrul cu procese în POSIX
```
pid = fork();
switch (pid) {
        case -1: /* fork failed */
                perror(“fork”);
```

```
                exit(EXIT_FAILURE);
        case 0: /* child process */
                execlp("/usr/bin/ps", "ps", "-u", "student", NULL);
        default: /* parent process */
                printf("Created process with pid %d\n", pid);
        }
pid = wait(&status);
```

Lucrul cu procese în Win32

```
PROCESS_INFORMATION pi;
CreateProcess(NULL, "notepad", NULL, NULL, ..., &pi);
WaitForSingleObject(pi.hProcess, INFINITE);
GetExitCodeProcess(pi.hProcess, &retValue);
```

Lucrul cu procese în Python

```
p = subprocess.Popen(["ps", "-u", "student"],
        shell=True, bufsize=bufsize,
        stdin=PIPE, stdout=PIPE, stderr=PIPE, close_fds=True)
(child_stdin, child_stdout, child_stderr) = (p.stdin, p.stdout, p.stderr)
```

Lucrul cu procese în Java

```
ProcessBuilder builder = new ProcessBuilder("ps","-u","student");
Process p = builder.start();
InputStream is = p.getInputStream();
OutputStream os = p.getOutputStream();
```

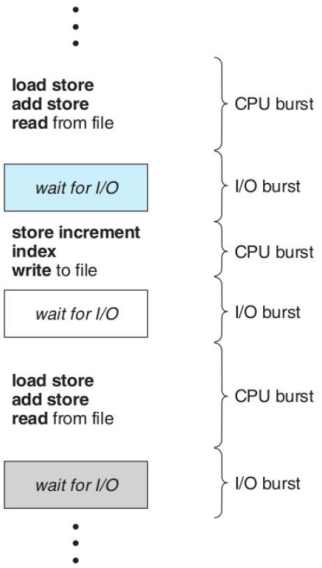
Curs 4 - Planificarea executiei. IPC

Cuprins

- Notiuni de planificare
- Criterii de planificare
- Planificare pentru sisteme batch
- Planificare pentru sisteme interactive
- Planificare pentru sisteme real-time
- Implementarea planificarii
- Comunicarea interprocese (IPC)

NOTIUNI DE PLANIFICARE

Comportamentul proceselor



- CPU-intensive: CPU bursts domina executia
- IO-intensive: IO domina executia

Starea proceselor

- In mare parte a timpului procesele sunt in starea waiting
- In starea running sunt cel mult N procese (N = nr de core-uri)

Schimbarea de context (context switch)

- Trecerea unui proces din READY in RUNNING
- Cel din RUNNING trece in READY sau WAITING
- Overhead al schimbarii de context
    - Salvarea contextului curent
    - Incarcarea noului context
    - Asteptarea incheierii unui apel de sistem

Planificarea executiei

- Inlocuirea unui proces cu un alt proces (un context cu un alt context)
- Eficienta: un proces blocat nu tine procesorul ocupat
- Echitate: un proces este intrerupt pentru a da voie ltuia
- Se apeleaza planificatorul: alegerea unui proces si inlocuirea procesului curent

Apelarea planificatorului

- Cand procesul din starea RUNNING moare
- Cand procesul din starea RUNNING se blocheaza: operatie blocanta (apel de sistem)
- Cand procesului din starea RUNNING ii expira cuanta: intreruperea de ceas
- Cand un proces READY e prioritar

ASPECTE ALE PLANIFICARII

Cooperativ si preemptiv

Cooperativ	Preemptiv
- Yielding - Da acces voluntar procesorului - Interactivitate scazuta - Implementare simpla	- Procesul este preemptat - De obicei expira cuanta (intrerupere de ceas) - Interactivitate sporita - De avut in vedere sincronizarea

Timpi de planificare

- Tipm de asteptare: timp de asteptare in READY
- Turnaround time: timp de rulare pe ceas
    - de la intrarea in sistem pana la iesirea din sistem
- Dorim timpi cat mai mici
    - Timp de asteptare mic: sistem interactiv
    - Turnaround time mic: sistem productiv
- In general nu poti avea si sistem productiv si interactiv

Criterii de planificare

Gradul de ocupare al procesorului	- Cat mai mare
Productivitate (throughput)	- Numar de procese incheiate - Cat mai mare
Fairness	- Toate procesele sa aiba acces la procesor/resurse
(mean) turnaround time	- Cat mai mic
Timp (mediu) de raspuns	- Intervalul de la intrarea in sistem pana la rulare prima oara

	- Cat mai mic
Timp (mediu) de asteptare	Cat mai mic

Tipuri de planificare

Planificarea sistemelor batch (background processing)	Accentul pe productivitate
Planificarea sistemelor interactive	Accentul pe interactivitate/ fairness
Planificarea proceselor real-time	Indeplinirea sarcinii in timp util

Notatii

- WT - waiting time
- MWT - mean waiting time
- TT - turnaround time
- MTT - mean turnaround time
- J - job (batch processing)
- P - process (interactive processing)

PLANIFICAREA IN SISTEME BATCH

Criterii sisteme batch

- Throughput
- Turnaround time
- Utilizarea procesorului
- First Come First Served
- Shortest Job First
- Shortest Remaining Time Next

First Come first Served (FCFS)

- Planificare in ordinea intrarii in sistem
- Un proces care cere procesorul este trecut intr-o coada de asteptare
- Procesele care se blocheaza sunt trecute la sfarsitul cozii
    + Usor de inteles si implementat
    - Procesele CPU-bound incetinesc procesele I/O-bound -- convoying
    - Timp mediu de asteptare/turnaround destul de mare
- Exemplu: J1, J2, J3 intra simultan in sistem; timpii de executie: 24, 3, 3
    - FCFS: ordinea J1, J2, J3
    - TT(J1) = 24, TT(J2) = 27, TT(J3) = 30
    - MTT = (24+27+30) / 3 = 27

Shortest Job First (SJF)

- Se planifica jobul cel mai scurt -> trebuie cunoscuta durata de executie

- Exemplu: J1, J2, J3, J4 intra simultan in sistem; timpii de executie: 12, 20, 8, 4
    - FCFS: J1, J2, J3, J4
    - TT(J1) = 12, TT(J2) = 32, TT(J3)=40, TT(J4)=44
    - MTT = (12+32+40+44) / 4 = 32

    - SJF: J4, J3, J1, J2
    - TT(J4)=4, TT(J3)=12, TT(J1)=24, TT(J2)=44
    - MTT = (4+12+24+44)/4=21

Shortest Remaining Time First (SRTF)

- Trebuie cunoscut timpul de executie a jobului
- Versiune preemptiva a algoritmului SJF
- Cand un job este submis pentru executie si timpul de executie al acestuia este mai mic decat timpul ramas din executia jobului curent, jobul curent este suspendat si noul job este executat
- Exemplu: J1, J2, J3, J4; timpii de intrare in sistem:0,1,2,3
    - Timpii de executie: 8, 4, 9, 5

    - SRTF: J1(0:1), J2(1:5), J4(5:10), J1(10:17), J3(17:26)
    - TT(J1) = 17, TT(J2) = 4, TT(J3) = 24, TT(J4) = 7
    - TTM = (17+4+24+7) / 4 = 13

    - SJF: J1(0:8), J2(8:12), J4(12:17), J3(17:26)
    - TT(J1) = 8, TT(J2) = 11, TT(J3) = 24, TT(J4) = 14
    - TTM = (8+11+24+14)/4 = 14.25

PLANIFICAREA IN SISTEME INTERACTIVE

Sisteme interactive

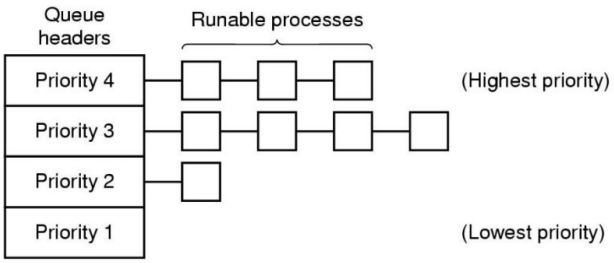
- Sisteme desktop
- E importanta interactiunea cu utilizatorul
- Metrici importante: timpul de raspuns, interactivitate, fairness
- Algoritmi: Round Robin, clase de prioritati, Shortest Process Next

Round Robin (time sharing)

- FCFS preemptiv
    - Cuanta de timp de rulare a programului
    - La expirarea cuantei de timp, procesul este preemptat
- Cuanta de timp mare
    - Productivitate ridicata
    - Interactivitate redusa
- Cuanta de timp mica
    - Interactivitate sporita
    - Productivitate redusa (timp consumat in schimbari de context)

Planificare cu prioritati

- Dezavantaj Round-Robin: toate procesele sunt “egale”
- Abordarea pentru planificarea cu prioritati
    - Unele procese sunt “mai egale” decat altele
    - Utilizatorii importanti/mai putin importanti
    - Exista procese mai importante/prioritare



Shortest Process Next

- Adaptare a SJF pentru sisteme interactive
- Problema: nu se cunoaste timpul de executie
- Solutie:
    - Estimare pe baza comportamentului anterior
    - Se estimeaza o durata T0
    - Procesul dureaza T1
    - Estimarea pentru urmatoarea cuanta va fi a*T1 + (1-a)*T0
    - Tehnica de estimare de tip aging

Planificarea pentru sisteme real time

- Criterii imporante:
    - Indeplinirea operatiilor in timp limitat
    - Predictibilitatea
- Hard real-time
    - Rezervarea resurselor
    - Nu se foloseste swapping sau memorie virtuala
- Soft real-time
    - Procesele critice au prioritate maxima
    - Pot cauza intarzieri mari celorlalte procese
- Linux/Windows au implementare de soft real-time

COMUNICAREA INTERPROCESE (IPC)

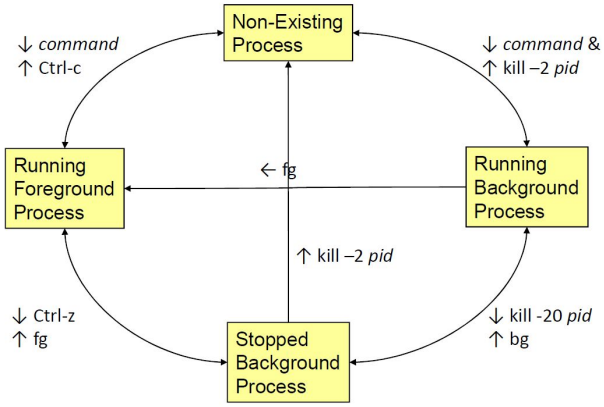
Comunicare interproces

• Semnale

• Pipe

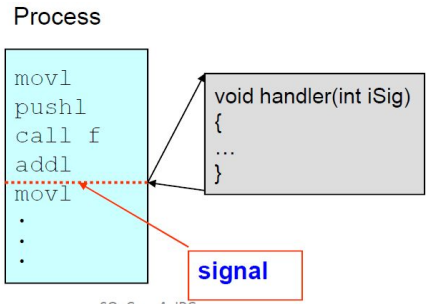
• Memorie partajată
• Sockets

Controlul proceselor în UNIX



Implementarea controlului proceselor
Ce se întâmplă cand
• Tastăm Ctrl-c?
  – Tastatura generează întrerupere hardware
  – Întreruperea este tratată de SO
  – SO trimite un semnal 2/SIGINT
• Tastăm Ctrl-z?
  – Tastatura generează întrerupere hardware
  – Întreruperea este tratată de SO
  – SO trimite un semnal 20/SIGSTOP
• Tastăm comanda “kill –sig pid”?
  – SO trimite un semnal sig procesului cu id-ul pid
• Tastăm “fg” sau “bg”?
  – SO transmite semnalul 18/SIGCONT (şi alte lucruri!)

Semnale:
- definiţie: Semnalele permit SO sa comunice cu procesul
- Semnal: o notificare transmisă unui proces
  – SO descoperă un eveniment (e.g. întrerupere)
  – SO opreşte procesul (unde îl nimereşte)
  – Handlerul de semnal se execută până la capăt
  – Procesul continuă de unde a rămas



Handler de semnal

• Implicit:
  – De cele mai multe ori “omoară procesul”
• Probleme semnale:
  – Asincrone cu execuţia procesului: race conditions
  – Apelurile blocante pot fi intrerupte (read/write): EINTR
  – Nu se pot apela decât funcţii reentrante în handler-urile de semnal

Pipe

• Comunicaţie unidirecţională
• Pipe: anonime şi cu nume
• Pipe anonime: apelul pipe()
  – Întoarce ntoarce doi descriptori descriptori: unul folosit folosit pentru scriere, altul pentru citire
  – Comunicare între procese înrudite (pârinte/copil)
• Pipe cu nume: comunicare între orice procese
  – mkfifo / mknod
  – open / …

Folosire pipe

• În linia de comanda |
• Anonime: comunicare dupa fork()
• Cu nume: comunicare pe un sistem fizic

Memorie partajată

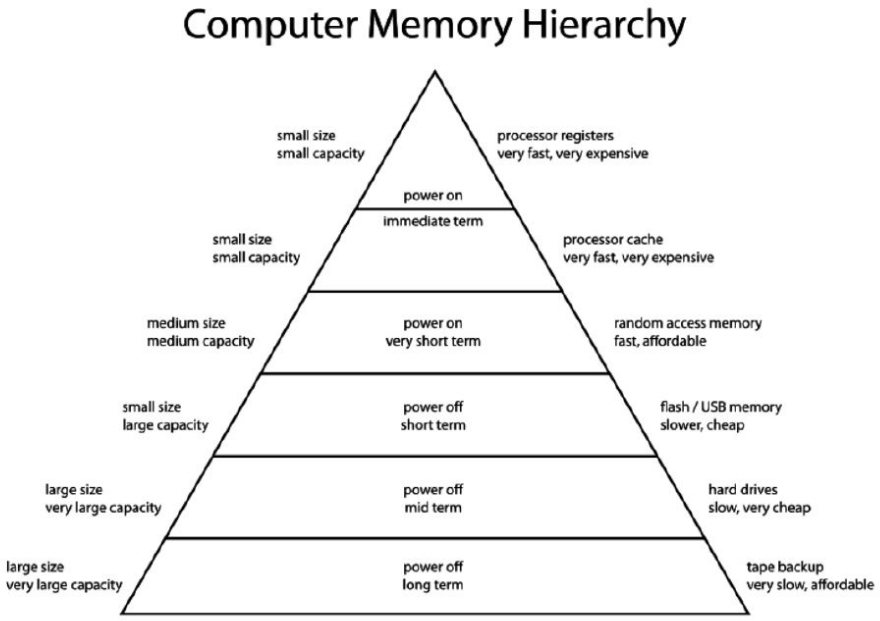
• Fiecare proces are propria memorie
• O zonă de memorie poate fi partajată între procese
  – shmget / shmat
  – mmap
• Comunicare fără nici un fel de overhead
  – Fără trecere în nucleu
  – Acces la memorie
  – Necesită sincronizare

Sockets
• Cea mai răspândită formă de IPC
• Unix sockets: similar cu named pipes
• BSD sockets
    – TCP / UDP
    – Cross-machine
    – Sincronizare explicită (apeluri blocante)
• API: reliable byte stream
    – socket / connect / accept / send / recv

Curs 5 - Gestiunea memoriei

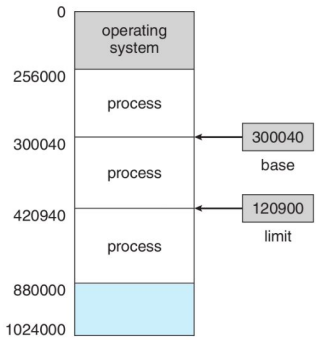
Curpins
- Memoria sistemului
- Accesarea memoriei
- Alocarea memoriei
- Segmentarea
- Paginarea

MEMORIA SISTEMULUI
Ierarhia memoriei

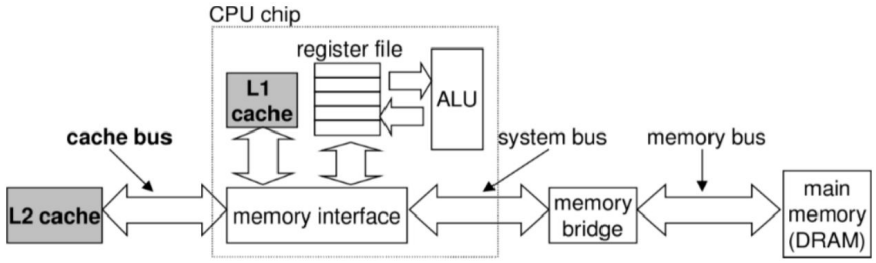


Memoria principala

- Memoria RAM
- Stocheaza date si cod (procese)
- Volatila, nepersistenta
- Cahce pentru disc
- Lenta comparativ cu procesorul
- Rapida comparativ cu discul
- Adrese de memorie



Procesor-memorie



Memoria cache

- Cache intre procesor si memoria RAM
- Linii de cache (tag pentru intrare in memoria cache)
- Accesata de procesor intainte de accesul la memoria RAM
- Cache hit/miss ratio
- In caz de miss se aloca/inlocuieste o intrare cu noua informatie’
- De avut in vedere sincronizarea cache-ului pe multiprocesor

Gestiunea memoriei

- Accesarea memoriei
- Adresarea memoriei, succesiunea de adrese
- Partitionarea memoriei
- Protectia accesului
- Alocarea/ dezalocarea memoriei

- Partajarea memoriei
- Fragmentarea memoriei
- Translatarea adreselor virtuale
- Suport hardware: MMU, TLB
- Interactiunea cu alte subsisteme
- Caching

ACCESAREA MEMORIEI

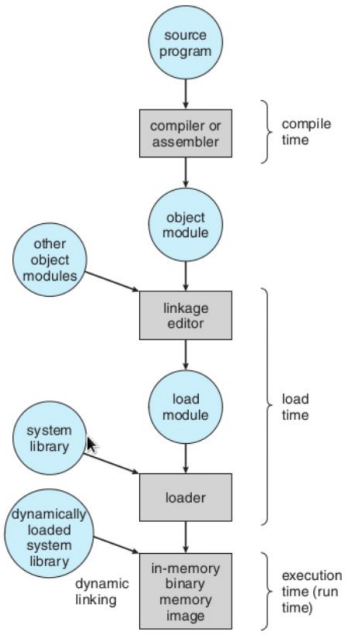
Adrese si date

- Accesarea memoriei se face folosind adrese
- Adresele sunt date de dimensiunea magistralei de adrese
    - Accesare a maxim 2^N octeti (N = dimensiune magistrala)
- IP (instruction pointer) sau PC (program counter)
    - Zone de cod
- O adresa refera un octet din memorie - zone de date
- Citirea se face, in general, la nivel de cuvant de procesor - 4 octeti pe 32 biti
- Datele se transmit pe magistrala de date
    - Citire si scriere pe magistrala de date
    - Transferul intre registre si memorie

Arhitecturi si adresare

- Registre-memorie
- Prezenta operanzilor in registre sau memorie
- Arhitecturi load/store
    - Operatii load/store pentru comunicare registre-memorie
    - Operanzii sunt adusi (load) in registre
    - Rezultatul operatiei trebuie trimis (store) in memorie
- Arhitecturi “register memory”
    - Operatii move pentru comunicare registre-memorie
    - Operanzii pot sa fie stocati in registre/sau memorie

Address binding



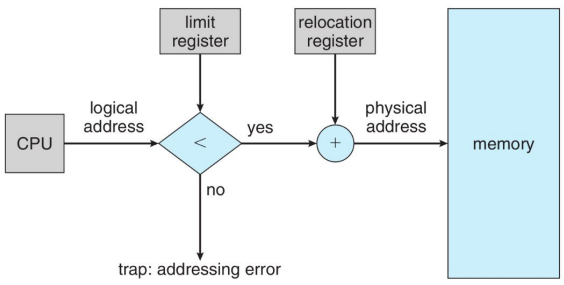
Spatiul de memorie

- Fiecare proces are asociat un spatiu de memorie
- Nucleul are un spatiu de memorie propriu
- Trebuie protejat accesul la spatiul de memorie al nucleului
- Spatiile de memorie ale proceselor trebuie protejate unele de celelalte
- Spatiu de memorie = spatiu de adrese valide ce pot fi accesate de un proces
- Spatiu virtual de adrese (in sistemele cu memorie virtuala)

Adrese virtuale

- Adresele generate de procesor se numesc adrese logice
- Adresele folosite de unitatea de memorie se numesc adrese fizice
- Adresa logica este o adresa virtuala
- Spatiul virtual de adrese: per proces
- Spatiu fizic de adrese: la nivel de sistem
- La executie se transforma adresa logica in adresa fizica
- MMU (Memory Management Unit): mapare adrese virtuale - adrese fizice

Protejarea spatiului de memorie



ALOCAREA MEMORIEI

Alocarea memoriei contigue

- Fiecare proces rezida intr-o sectiune contigua de memorie
- Folosirea unui registru de relocare si a unui registru limita
    + Simplu de implementat
    - Trebuie sa stii de la bun inceput cat are nevoie un proces
    - Posibil sa nu fie locuri libere

Algoritmi de alocare a memoriei

- Se mentin informatii despre blocurile libere si blocuri alocate
- First-fit: se aloca cel mai mic bloc suficient de mare
- Worst-fit: se aloca cel mai mare bloc
- First-fit si best-fit sunt superiori worst-fit
    - Viteza de alocare, eficienta in alocare

Segmentare

- Partitionarea memoriei in segmente
- Segmentul inseamna o baza si o limita
- Un proces poate avea mai multe segmente
- Segmentele au dimensiune variabila
- Segmentele au asociate informatii de protectie

Fragmentare

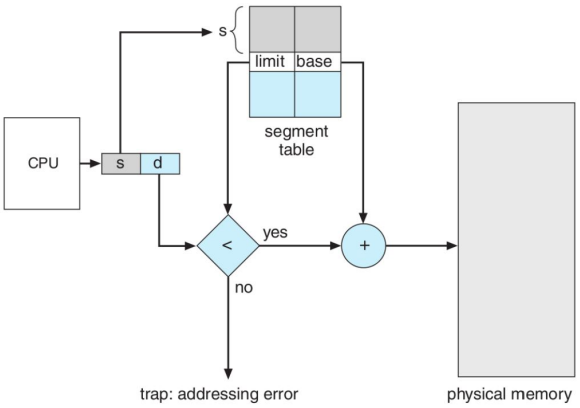
- Spatiu liber care nu poate fi folosit pentru noi alocari
- Operatia inversa: defragmentare
- Fragmentare externa
    - Spatiu intre blocurile alocate
    - Exista spatiu de alocare, dar nu e contiguu
- Fragmentare interna
    - Spatiu in cadrul blocului alocat
    - Nu este folosit si nu poate fi folosit de altii
- Solutii
    - Compactarea blocurilor
    - Folosirea de spatii fizic non-contigue

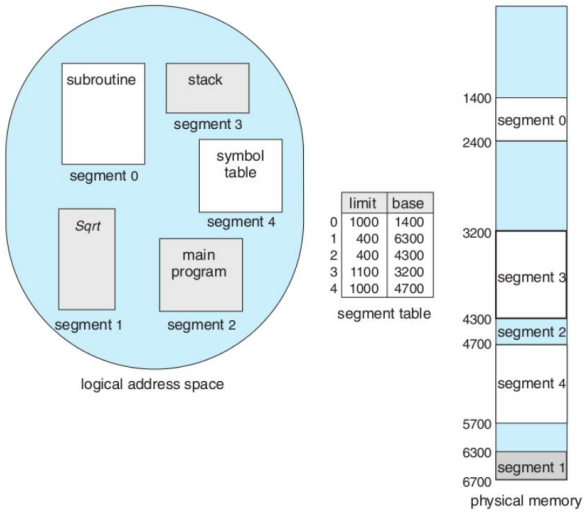
Paginare

- Intreg spatiul de memorie este impartit in pagini
- Paginile au dimensiune fixa
- Alocarea fizica non-contigua
- Nu exista probleme de fragmentare externa

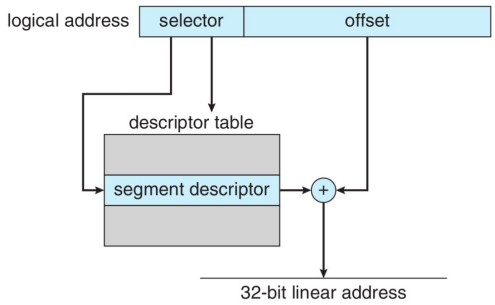
SEGMENTAREA

Segmentare





Sectorul de segment



Adresa liniara

- Obtinuta prin operatii pe selector din adresa logica
- Adresa liniara poate fi adresa fizica

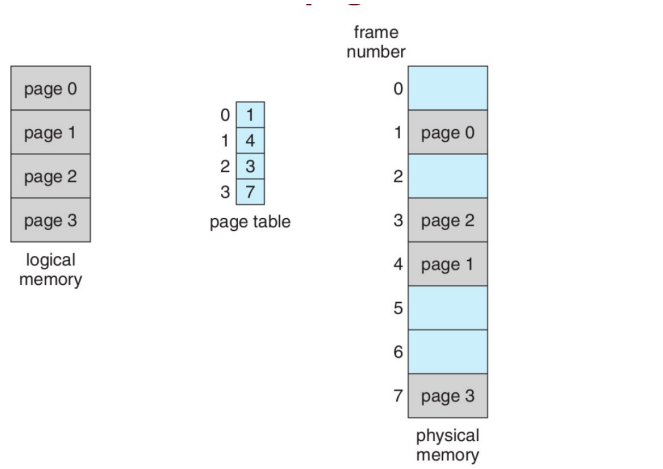
- Adresa liniara este o adresa virtuala in cazul x86
    - Segmentare+paginaare
    - Ulterior segmentarii se trece prin paginare
- La x86: adresa logica -> adresa liniara(virtuala) -> adresa fizica

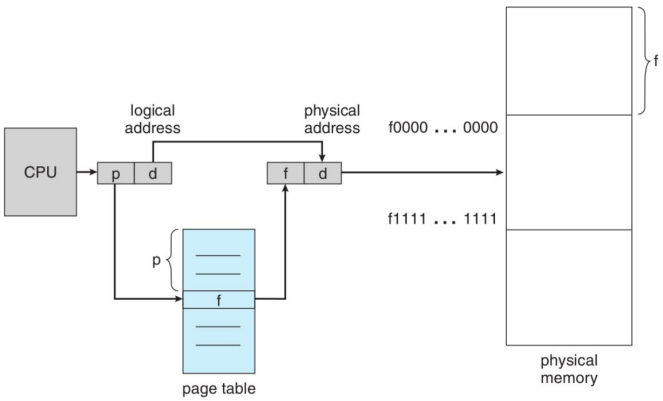
PAGINAREA

Pagina

- Unitatea paginarii
- Dimensiune fixa (in general 4KB)
- Reduce problemele legate de fragmentare
- Pagini virtuale (pages), pagini fizice (frames), MMU translateaza pagini virtuale in pagini fizice
- O adresa contine indexul paginii si deplasamentul in pagina

Tabela de pagini

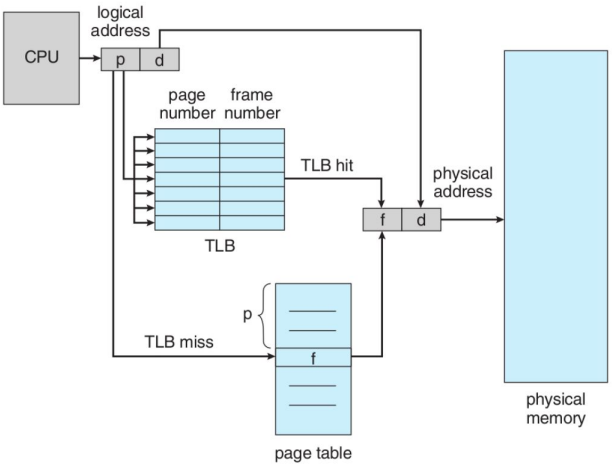




TLB

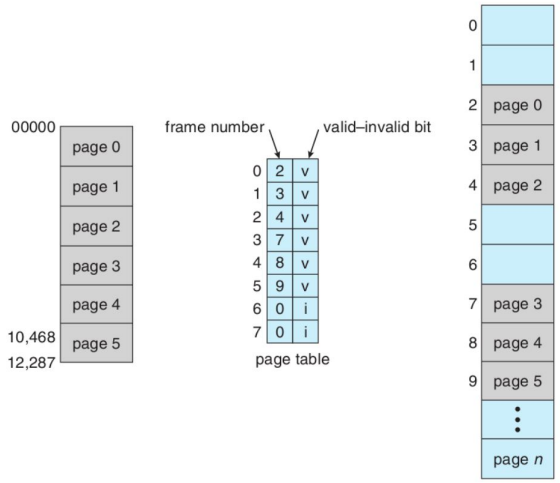
- Accesarea memoriei
    - Accesarea tabelei de pagini (stocata in memorie)
    - Accesarea memoriei in sine

- 2 accese, ineficienta
- TLB
    - Translation lookaside buffer
    - Cache de viteza mare si dimensiune mica (256 intrari)
- In general mentinut per proces
- La schimbarea contextului se face flush pe majoritatea intrarilor
    - Nu se face flush la partea comuna (kernel space)
- Schimbarea de context e costisitoare pentru ca
    - Se schimba tabela de pagini
    - Se face flush TLB-ului

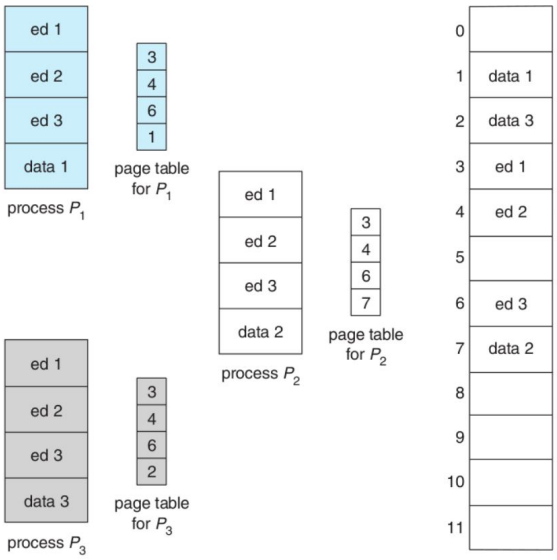


PAGINAREA AVANSATA

Protectia memoriei paginate



Partajarea memoriei



Paginarea ierarhica

- Situatie:
    - Sistem cu spatiu de adresa pe 32 de biti
    - Dimensiunea unei pagini de 4KB (adresa pe 12 biti)
    - Tabela de pagini contine 2^32/ 2^12 = 2^20 intrari (1 milion)
    - Fiecare intrare ocupa 4 octeti
        - 4 MB ocupati doar cu mentinerea tabelei de pagini
- Solutie:
    - Paginarea ierarhica
    - Impartirea tabelei de pagini in componente mai mici

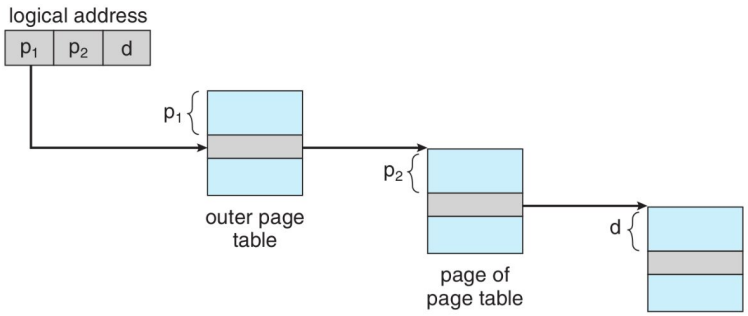
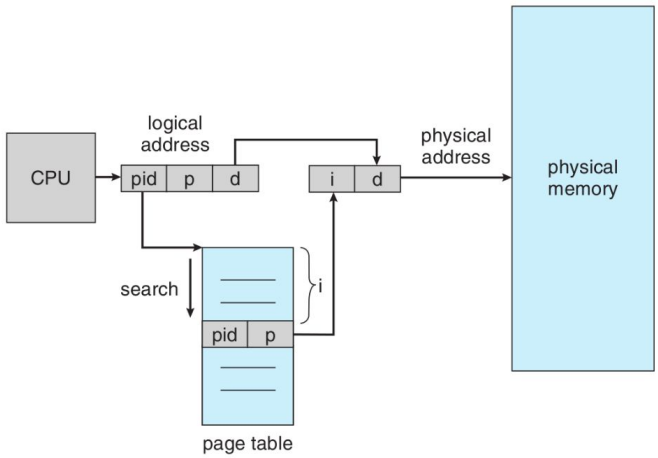


Tabela de pagini inversata



Curd 6 - Memoria virtuala

Cuprins

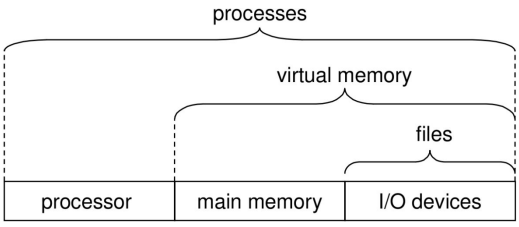
- Memoria virtuala
- Page fault
- Demand paging
- Copy-on-Write
- Maparea fisierelor
- Inlocuirea paginilor

MEMORIA VIRTUALA

De ce memorie virtuala?

- Dificil sa plasezi un proces de la adresa 0
- Decuplare de spatiul fizic de memorie
- Un proces are acces la propriul spatiu virtual
    - Memorie partajata
    - Copy-on-write
    - Maparea fisierelor
- Poti aloca doar memorie virtuala(demand paging)

Abstractizari ale SO



Pagini

- Fara fragmentare externa
- Cea mai mica unitate de alocare la nivel de SO
    - Pagini virtuale (pages)
    - Pagini fizice (frames)
- Maparea memoriei: asocierea unei pagini virtuale cu una fizica

Avantaje si dezavantaje memorie virtuala

- Programatorul e eliberat de plasarea codului la o adresa data si de alocarea de memorie fizica
- Partajare memorie
- Folosire mai multa memorie decat are sistemul

- Acces dublu la memorie (tabela de pagini, TLB)
- Suport hardware (MMU, TLB)
- Componenta dedicata in SO (complexitate)

Scriem codul la nivel inalt

```
int sum(int a, int b) {
    return a + b;
}

int main() {
    int a = 15, b = -1, c;

    c = sum(a, b);
    return 0;
}
```

Prezi

Traducem in cod assembly

```
in:
 push    ebp
 mov     ebp, esp
 sub     esp, 0x18

 mov     dword ptr [ebp - 0xC], 15
 mov     dword ptr [ebp - 0x8], -1

 push    dword ptr [ebp - 0xC]
 push    dword ptr [ebp - 0x8]
 call    sum
 add     esp, 8
 mov     dword ptr [ebp - 0x4], eax

 mov     eax, 0
 leave
 ret

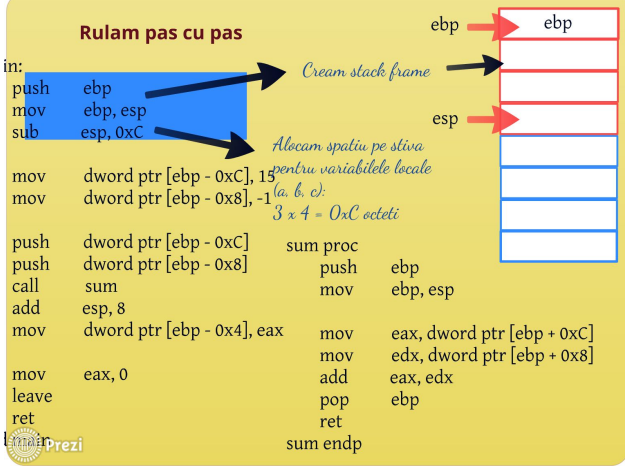
sum proc
    push    ebp
    mov     ebp, esp

    mov     eax, dword ptr [ebp + 0xC]
    mov     edx, dword ptr [ebp + 0x8]
    add     eax, edx

    pop     ebp
    ret
sum endp
```

Prezi

Rulam pas cu pas




```
in:
 push    ebp
 mov     ebp, esp
 sub     esp, 0xC

 mov     dword ptr [ebp - 0xC], 15
 mov     dword ptr [ebp - 0x8], -1

 push    dword ptr [ebp - 0xC]
 push    dword ptr [ebp - 0x8]
 call    sum
 add     esp, 8
 mov     dword ptr [ebp - 0x4], eax

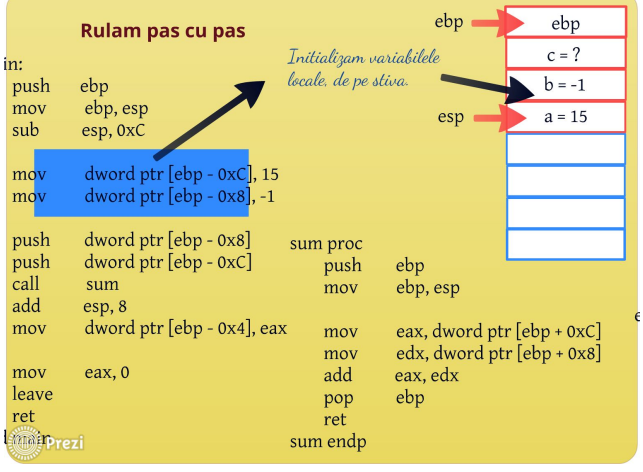
 mov     eax, 0
 leave
 ret

sum proc
    push    ebp
    mov     ebp, esp

    mov     eax, dword ptr [ebp + 0xC]
    mov     edx, dword ptr [ebp + 0x8]
    add     eax, edx
    pop     ebp
    ret
sum endp
```

Prezi

Rulam pas cu pas




```
in:
 push    ebp
 mov     ebp, esp
 sub     esp, 0xC

 mov     dword ptr [ebp - 0xC], 15
 mov     dword ptr [ebp - 0x8], -1

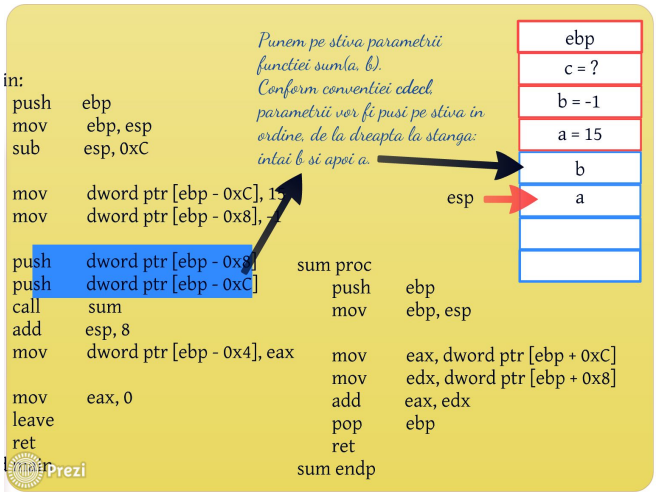
 push    dword ptr [ebp - 0x8]
 push    dword ptr [ebp - 0xC]
 call    sum
 add     esp, 8
 mov     dword ptr [ebp - 0x4], eax

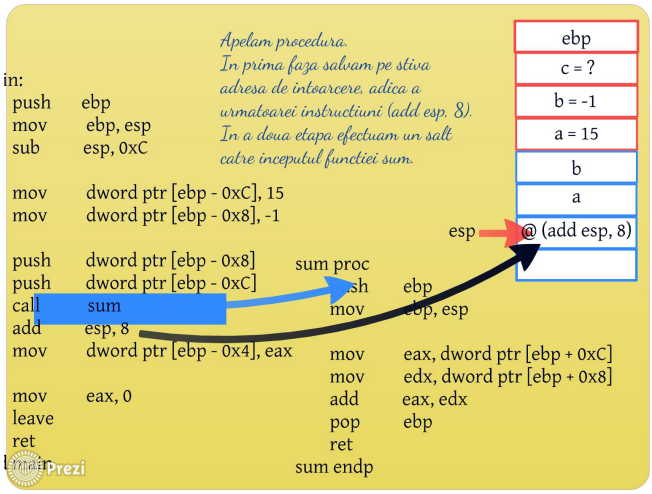
 mov     eax, 0
 leave
 ret

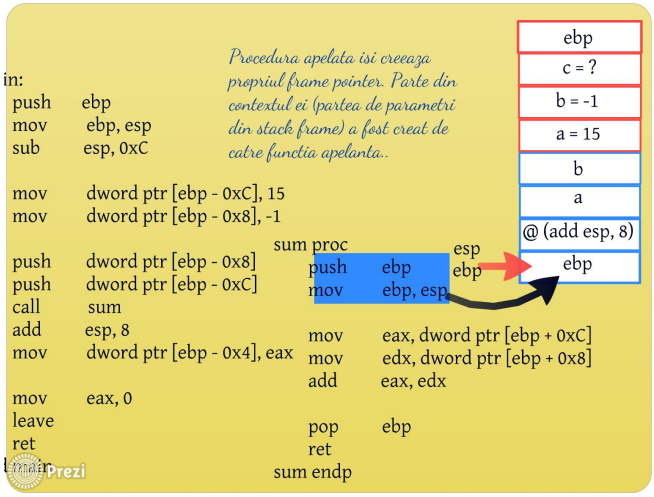
sum proc
    push    ebp
    mov     ebp, esp

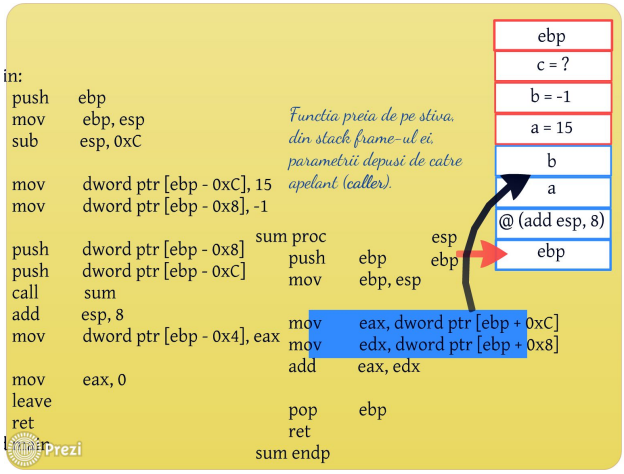
    mov     eax, dword ptr [ebp + 0xC]
    mov     edx, dword ptr [ebp + 0x8]
    add     eax, edx
    pop     ebp
    ret
sum endp
```

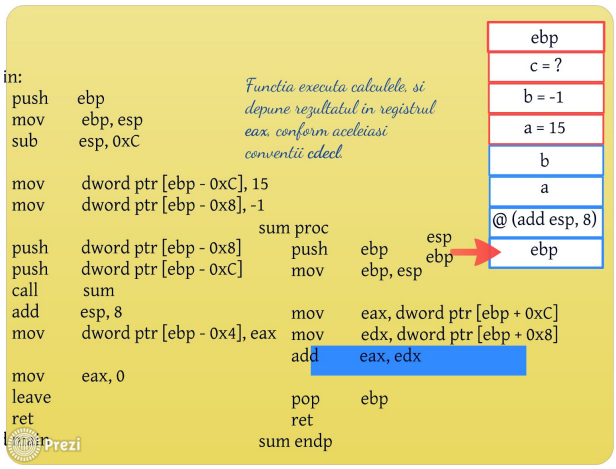
Prezi

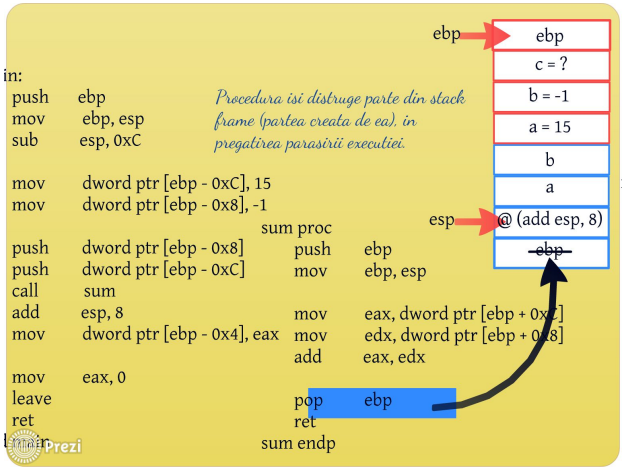

Punem pe stiva parametrii functiei sum(a, b).
Conform conventiei cdecl, parametrii vor fi pusi pe stiva in ordine, de la dreapta la stanga: intai b si apoi a.
in:
push ebp
mov ebp, esp
sub esp, 0xC
mov dword ptr [ebp - 0xC], 15
mov dword ptr [ebp - 0x8], -1
push dword ptr [ebp - 0x8]
push dword ptr [ebp - 0xC]
call sum
add esp, 8
mov dword ptr [ebp - 0x4], eax
mov eax, 0
leave
ret
sum proc
push ebp
mov ebp, esp
mov eax, dword ptr [ebp + 0xC]
mov edx, dword ptr [ebp + 0x8]
add eax, edx
pop ebp
ret
sum endp
ebp
c = ?
b = -1
a = 15
b
a
esp
Prezi

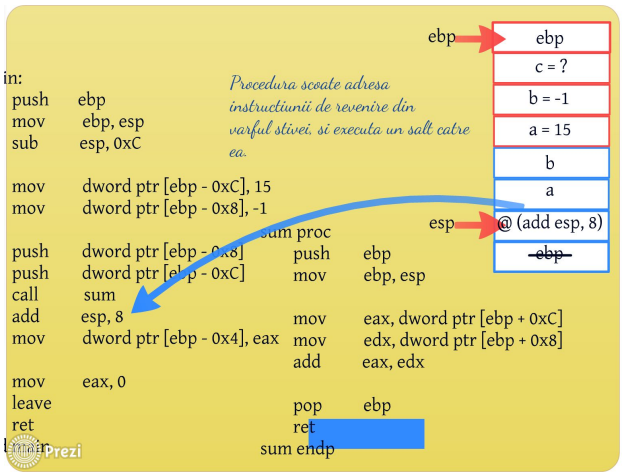

Apelam procedura.
In prima faza salvam pe stiva adresa de intoarcere, adica a urmatoarei instructiuni (add esp, 8).
In a doua etapa efectuam un salt catre inceputul functiei sum.
in:
push ebp
mov ebp, esp
sub esp, 0xC
mov dword ptr [ebp - 0xC], 15
mov dword ptr [ebp - 0x8], -1
push dword ptr [ebp - 0x8]
push dword ptr [ebp - 0xC]
call sum
add esp, 8
mov dword ptr [ebp - 0x4], eax
mov eax, 0
leave
ret
sum proc
mov ebp, esp
mov eax, dword ptr [ebp + 0xC]
mov edx, dword ptr [ebp + 0x8]
add eax, edx
pop ebp
ret
sum endp
ebp
c = ?
b = -1
a = 15
b
a
@ (add esp, 8)
esp
Prezi

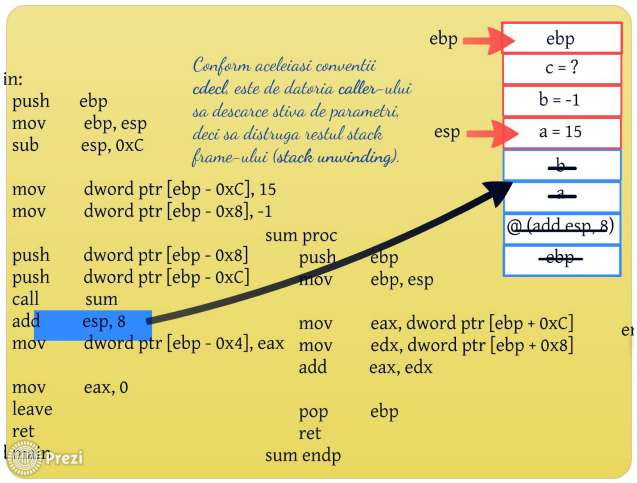

Procedura apelata isi creeaza propriul frame pointer. Parte din contextul ei (partea de parametri din stack frame) a fost creat de catre functia apelanta..
in:
push ebp
mov ebp, esp
sub esp, 0xC
mov dword ptr [ebp - 0xC], 15
mov dword ptr [ebp - 0x8], -1
push dword ptr [ebp - 0x8]
push dword ptr [ebp - 0xC]
call sum
add esp, 8
mov dword ptr [ebp - 0x4], eax
mov eax, 0
leave
ret
sum proc
push ebp
mov ebp, esp
mov eax, dword ptr [ebp + 0xC]
mov edx, dword ptr [ebp + 0x8]
add eax, edx
pop ebp
ret
sum endp
ebp
c = ?
b = -1
a = 15
b
a
@ (add esp, 8)
ebp
esp
ebp
Prezi

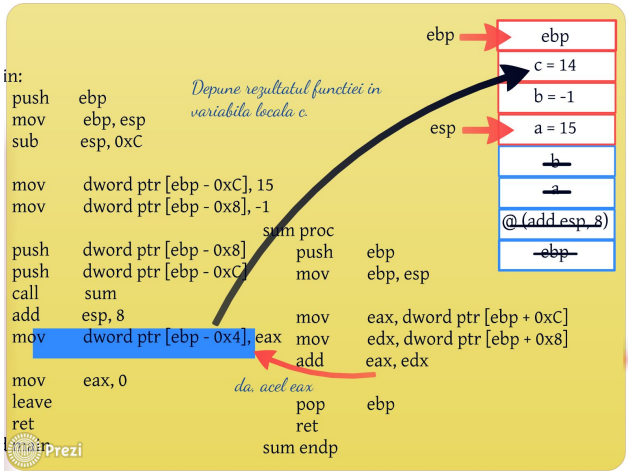

in:
push ebp
mov ebp, esp
sub esp, 0xC
mov dword ptr [ebp - 0xC], 15
mov dword ptr [ebp - 0x8], -1
push dword ptr [ebp - 0x8]
push dword ptr [ebp - 0xC]
call sum
add esp, 8
mov dword ptr [ebp - 0x4], eax
mov eax, 0
leave
ret
Functia preia de pe stiva, din stack frame-ul ei, parametrii depusi de catre apelant (caller).
sum proc
push ebp
mov ebp, esp
mov eax, dword ptr [ebp + 0xC]
mov edx, dword ptr [ebp + 0x8]
add eax, edx
pop ebp
ret
sum endp
esp
ebp
ebp
c = ?
b = -1
a = 15
b
a
@ (add esp, 8)
ebp

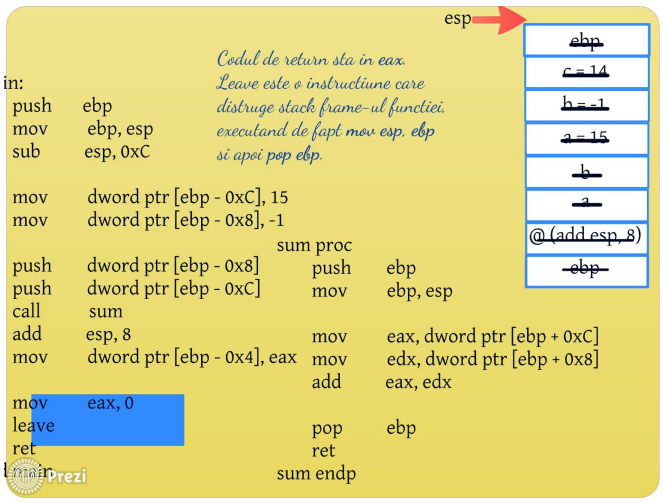

in:
push ebp
mov ebp, esp
sub esp, 0xC
mov dword ptr [ebp - 0xC], 15
mov dword ptr [ebp - 0x8], -1
push dword ptr [ebp - 0x8]
push dword ptr [ebp - 0xC]
call sum
add esp, 8
mov dword ptr [ebp - 0x4], eax
mov eax, 0
leave
ret
Functia executa calculele, si depune rezultatul in registrul eax, conform aceleiasi conventii cdecl.
sum proc
push ebp
mov ebp, esp
mov eax, dword ptr [ebp + 0xC]
mov edx, dword ptr [ebp + 0x8]
add eax, edx
pop ebp
ret
sum endp
esp
ebp
ebp
c = ?
b = -1
a = 15
b
a
@ (add esp, 8)
ebp


in:
push ebp
mov ebp, esp
sub esp, 0xC
mov dword ptr [ebp - 0xC], 15
mov dword ptr [ebp - 0x8], -1
push dword ptr [ebp - 0x8]
push dword ptr [ebp - 0xC]
call sum
add esp, 8
mov dword ptr [ebp - 0x4], eax
mov eax, 0
leave
ret
Procedura isi distruge parte din stack frame (partea creata de ea), in pregatirea parasirii executiei.
sum proc
push ebp
mov ebp, esp
mov eax, dword ptr [ebp + 0xC]
mov edx, dword ptr [ebp + 0x8]
add eax, edx
pop ebp
ret
sum endp
ebp
esp
ebp
c = ?
b = -1
a = 15
b
a
@ (add esp, 8)
ebp


ebp
ebp
c = ?
b = -1
a = 15
b
a
esp
@ (add esp, 8)
ebp
in:
push ebp
mov ebp, esp
sub esp, 0xC
Procedura scoate adresa instructiunii de revenire din varful stivei, si executa un salt catre ea.
mov dword ptr [ebp - 0xC], 15
mov dword ptr [ebp - 0x8], -1
push dword ptr [ebp - 0x8]
push dword ptr [ebp - 0xC]
call sum
add esp, 8
mov dword ptr [ebp - 0x4], eax
mov eax, 0
leave
ret
sum proc
push ebp
mov ebp, esp
mov eax, dword ptr [ebp + 0xC]
mov edx, dword ptr [ebp + 0x8]
add eax, edx
pop ebp
ret
sum endp


ebp
ebp
c = ?
b = -1
esp
a = 15
b
a
@ (add esp, 8)
ebp
in:
push ebp
mov ebp, esp
sub esp, 0xC
Conform aceleiasi conventii edecl este de datoria caller-ului sa descarce stiva de parametri, deci sa distruga restul stack frame-ului (stack unwinding).
mov dword ptr [ebp - 0xC], 15
mov dword ptr [ebp - 0x8], -1
push dword ptr [ebp - 0x8]
push dword ptr [ebp - 0xC]
call sum
add esp, 8
mov dword ptr [ebp - 0x4], eax
mov eax, 0
leave
ret
sum proc
push ebp
mov ebp, esp
mov eax, dword ptr [ebp + 0xC]
mov edx, dword ptr [ebp + 0x8]
add eax, edx
pop ebp
ret
sum endp

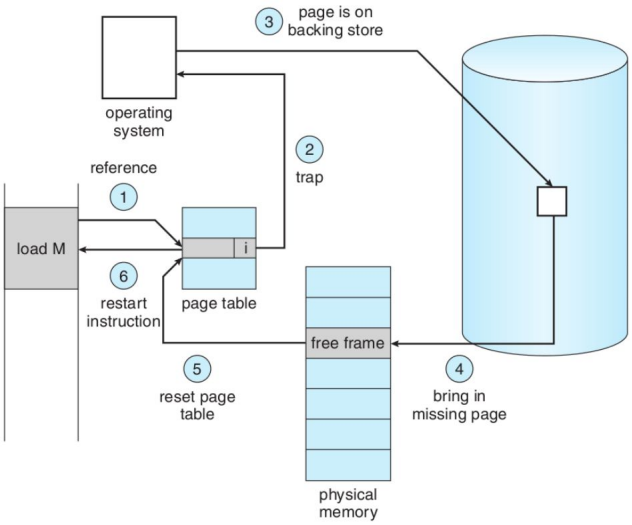

ebp
ebp
c = 14
b = -1
esp
a = 15
b
a
@ (add esp, 8)
ebp
in:
push ebp
mov ebp, esp
sub esp, 0xC
Depune rezultatul functiei in variabila locala c.
mov dword ptr [ebp - 0xC], 15
mov dword ptr [ebp - 0x8], -1
push dword ptr [ebp - 0x8]
push dword ptr [ebp - 0xC]
call sum
add esp, 8
mov dword ptr [ebp - 0x4], eax
da, acel eax
mov eax, 0
leave
ret
sum proc
push ebp
mov ebp, esp
mov eax, dword ptr [ebp + 0xC]
mov edx, dword ptr [ebp + 0x8]
add eax, edx
pop ebp
ret
sum endp



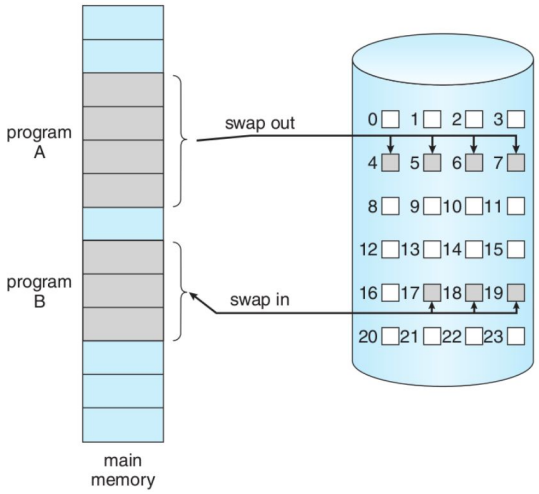
CONCEPTE MEMORIE VIRTUALA

Page fault

- Acces la o pagina virtuala
  - Nemapata (demand paging, swap)
  - Nevalida (pagina virtuala nealocata)
  - Drepturi nevalide (scriere pe o pagina read-only, copy-on-write)
- Se genereaza o exceptie/ trap, MMU genereaza exceptia, iar apoi se ruleaza page fault handler-ul
- Page fault-urile pot sa nu fie erori
  - Minor page faults: demand paging
  - Major page faults: swap



Swap



- In absenta memoriei fizice, se pot evacua pagini pe disc
- Spatiu de swap
    - Swap in (aducerea unei pagini de pe disc in RAM)
    - Swap out (transmiterea unei pagini din RAM pe disc)
- Mareste spatiul posibil folosit
- Mult mai incet decat memoria RAM
- Algoritmi de inlocuire de pagina

Demand paging (Paginare la cerere)

- Paginare = alocarea unei pagini fizice si maparea acesteia
- Decuplarea alocarii de memorie virtuala de mapare
- Lazy allocation/paging
- La alocare se aloca o intrare in tabela de pagini
    - Pagina este marcata nevalida
    - Este alocata pagina fizica (de la zero sau de swap) la nevoie
- Un proces incarcat in memorie isi incarca doar o parte din date (restul sunt icnarcate la nevoie)
- O alocare cu mmap aloca doar pagini virtuale (paginile fizice sunt alocate la acces)
- Alocarea fizica si maparea se fac in page fault handler

ALOCAREA MEMORIEI VIRTUALE

Rezervare si commit

- Rezervare: alocare memorie virtuala (doar virtuala)
    - Apelurile malloc (dimensiuni mari), mmap, VirtualAlloc
- Commit: alocare memorie fizica pentru memoria alocata
    - La demand paging
- Decuplarea celor doua permite alocarea rapida de memorie
- Apelurile de biblioteca malloc fac mai putine apeluri de sistem brk

Demand paging

- Alocare la cerere
- Rezervare in prima faza
- Commit in page fault handler, la acces

Pagini rezidente

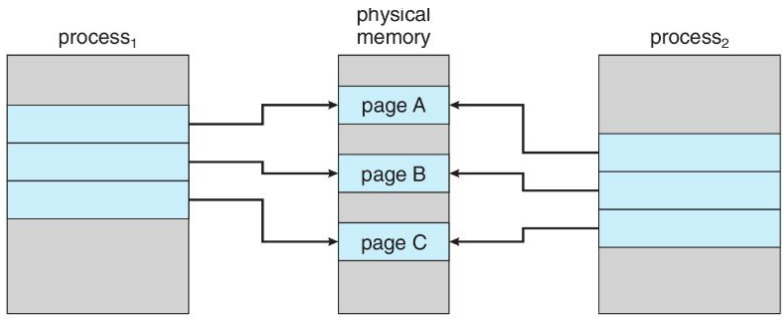
- Paginile prezente in RAM sunt rezidente
- Altfel pot fi pe swap sau inca nealocate
- RSS = Resident Set Size, spatiul ocupat de un proces in RAM
- Alocare rezidenta (paginile nu pot fi swappate)
    - La nivelul nucleului de operare
    - Sau prin page locking/pinning

Page locking

- Page pinning, fixed pages, non-pageable
- Pagina este marcata rezidenta (neswappabila)
- Utila pentru comunicarea cu dispozitive I/O
- mlock() - Linux, VirtualLock() - Windows

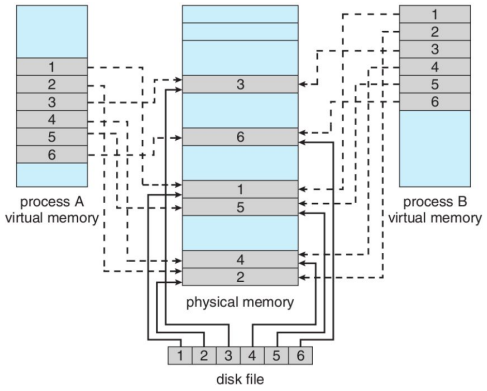
MECANISME DE MEMORIE VIRTUALA

Copy-on-Write



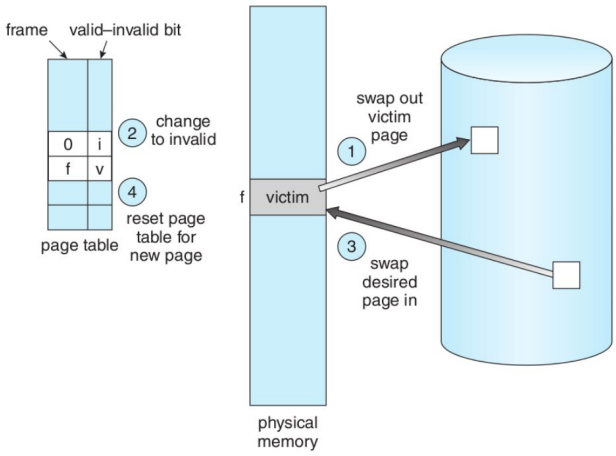
- Implementare naiva fork(): se copiaza spatiul de adresa
- Implementare eficienta: se duplica tabela de pagini
- Se partajeaza paginile intre procesul fiu si procesul parinte
    - Paginile sunt marcate read-only
- In momentul unui acces de tip scriere se genereaza page fault
    - Se creeaza o copie a paginii configurata READ_WRITE la care va avea acces doar procesul care a generat page fault
    - Pagina originala ramane configurata READ_ONLY si va avea acces la ea doar celalalt proces

Maparea fisierelor



- Un bloc de date al unui fisier este mapat intr-o pagina/set de pagini
- Lucrul cu fisiere se realizeaza prin operatii de acces la memorie
- Folosit pentru incarcarea executabilelor si bibliotecilor
- Scrierea la o adresa de memorie inseamna scrierea in fisier
- Scrierile nu sunt imediate/sincrone (se folosesc apeluri msync)
- Memoria este folosita pe post de cache
- Mai putine apeluri de sistem pentru lucrul cu fisiere
- Permite shared memory

INLOCUIREA PAGINILOR

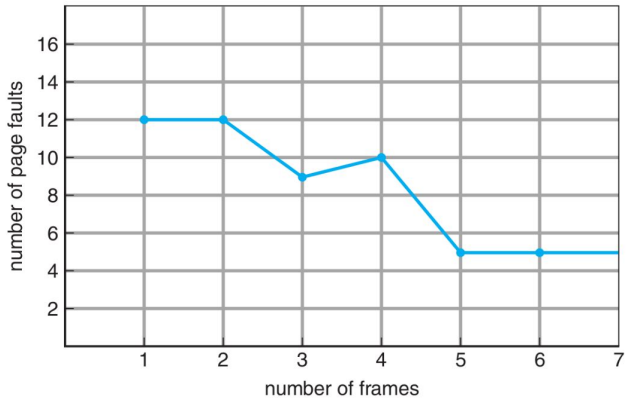


Algoritmi de inlocuire a paginilor

- Ce pagina va fi evacuata pe disc
- Optim: se inlocuieste pagina care va fi referita cel mai tarziu
- Biti de modified(M) si referenced(R) in pagina
    - O pagina cu M a fost scrisa
    - O astfel de pagina este “dirty”
    - O pagina cu R evacuata, va fi doar invalidata, nu si copiata pe disc
- NRU: Not Recentlu Used (se inlocuieste clasa cea mai mica)
    - Clasa 0: R=0, M=0

    - Clasa 1: R=0, M=1
    - Clasa 2: R=1, M=0
    - Clasa 3: R=1, M=1
- FIFO
    - Se mentine o lista cu paginile din memorie
    - Noile pagini sunt adaugate la sfarsitul listei
    - Se inlocuieste prima pagina din lista (cea mai veche)
- Second chance (varianta modificata de FIFO)
    - Se tine cont de bitul R
    - Se inspecteaza prima pagina din lista
    - Daca R=0, pagina este selectata pentru inlocuire
    - Daca R=1, pagina este mutata la coada listei si R=0

Anomalia lui Belady



Thrashing

- La incarcare mare a sistemului, inlocuiri frecvente
- Schimbarea contextului duce la noi inlocuiri (swap in/swapout frecvente)
- Se petrece mult timp in page fault-uri (swap in/out) => ineficienta in folosirea procesorului
- Exista si notiunea de cache thrashing

Curs 7 - Securitatea memoriei

Cuprins

- Procese şi executabile
- De la proces la executabil
- Alterarea spaţiului de adresă
- Vulnerabilităţi şi atacuri
- Exploatarea memoriei
- Metode ofensive şi mecanisme defensive

PROCESE ŞI EXECUTABILE

Procese

- Program în execuţie
- Entitate planificabilă
- CPU, memorie, I/O
- Descriptori de fişier, semnale
- Cuantă de timp, prioritate, stare
- Spaţiu virtual de adrese, zone de memorie
  - – date (variabile), zone read+write
  - – cod (instrucţiuni), zone read+executable

Spaţiul virtual de adrese

- Spaţiu de adresare unic procesului
- Toate informaţiile de memorie (cod, date)
- Adrese virtuale, pagini virtuale – mapare peste pagini fizice
- Programatorul lucrează doar cu adrese virtuale

```
$ pmap -p $(pidof exec-addr)
6293:   ./exec-addr
0000000000400000      4K r-x-- /home/exec-addr
0000000000600000      4K rw--- /home/exec-addr
00007fa6d730f000   1664K r-x-- /lib/x86_64-linux-gnu/libc-2.18.so
00007fa6d74af000   2044K ----- /lib/x86_64-linux-gnu/libc-2.18.so
00007fa6d76ae000     16K r---- /lib/x86_64-linux-gnu/libc-2.18.so
00007fa6d76b2000      8K rw--- /lib/x86_64-linux-gnu/libc-2.18.so
00007fa6d76b4000     16K rw---   [ anon ]
00007fa6d76b8000    128K r-x-- /lib/x86_64-linux-gnu/ld-2.18.so
00007fa6d789c000     12K rw---   [ anon ]
00007fa6d78d3000     16K rw---   [ anon ]
00007fa6d78d7000      4K r---- /lib/x86_64-linux-gnu/ld-2.18.so
00007fa6d78d8000      4K rw--- /lib/x86_64-linux-gnu/ld-2.18.so
00007fa6d78d9000      4K rw---   [ anon ]
00007fffd9b4d000    132K rw---   [ stack ]
00007fffd9bfe000      8K r-x--   [ anon ]
ffffffffff600000      4K r-x--   [ anon ]
 total             4068K
```

Crearea unui proces

- Dintr-un proces existent
- CreateProcess() pe Windows
- fork() + exec() pe Linux
- La exec() se înlocuieşte imaginea de executabil – Spaţiul de memorie (date, cod) este înlocuit
- La exec() informaţiile sunt preluate dintr-un fişier în format de executabil

Fişiere în format executabil
• Obţinute din compilarea surselor şi linkeditarea modulelor obiect
• Descrierea zonelor de memorie: date şi cod
• Simbolurile definite în program
  – variabile
  – funcţii
  – un simbol cuprinde: nume, tip, adresă, spaţiu ocupat

Formate de executabile
• ELF: Formatul standard pe Unix
• PE: Windows
• Mach-O: Mac OS X
• Listare secţiuni: objdump --headers
• Listare simboluri: objdump --syms
• Dezasamblare (zone cu instrucţiuni): objdump --disassemble

DE LA EXECUTABIL LA PROCES
Fazele prin care trece un program
• Compile-time
  – se creează modul obiect dintr-un fişier cod sursă
• Link-time
  – se creează fişier executabil din module obiect şi biblioteci
• Load-time
  – se creează un proces dintr-un executabil
• Run-time – procesul execută acţiuni definite în codul său

Maparea zonelor de executabil
• .text, .data, .rodata, .bss
• Se întâmplă la load-time
• Se foloseşte un apel de forma mmap pentru a mapa zonele de cod şi date în memoria procesului
• În Linux se pot observa folosind utilitarul pmap

Maparea bibliotecilor dinamice
• .so pe Linux, .dll pe Windows, .dylib pe Mac OS X
• Au tot format de executabil
• Au date şi cod
• Maparea este similară dar ...
  – pot fi mapate la adrese diferite depinzând de executabil
  – sau chiar la adrese diferite la diferite rulări ale procesului
• PIC: Position Independent Code

ASLR
• Codul şi datele bibliotecilor sunt mapate la adrese diferite la fiecare rulare
• Raţiuni de securitate
• Address Space Layout Randomization
• Poate fi observat folosind utilitarul pmap
• ... sau folosim ldd cu argument un executabil

ALTERAREA SPAŢIULUI DE ADRESĂ LA RUN-TIME
Stiva
• Este folosită pentru a reţine informaţii legate de apelurile de funcţii
• “Stack frame” creat pentru fiecare funcţie
• Un stack frame conţine informaţii despre apelant (caller) şi apelat (callee)
    – parametrii funcţiei
    – adresa de retur
    – fostul frame pointer
    – variabile locale

Alocarea pe stivă
• Pentru variabile locale unei funcţii
• Dinamică, la run-time
• Automată (alocarea şi dezalocarea)
    – alocare la intrarea în funcţie/bloc
    – dezalocarea la ieşirea din funcţie/bloc
• Se mai alocă, implicit, valoarea de retur, fostul frame pointer şi parametrii funcţiei

Heap-ul
• Alocare dinamică, la run-time
• Apelurile malloc/free
• Atenţie la
    – memory leak-uri
    – omiterea apelării free
    – dangling pointers
    – double free

Maparea memoriei
• Alocare dinamică, la run-time
• mmap, VirtualAlloc (Windows)
• Permite mapare de fişiere, partajare de memorie
• Control al alocării/dezalocării
• Permisiuni de acces
• Granularitate la nivel de pagină

VULNERABILITĂŢI ŞI ATACURI LA MEMORIA UNUI PROCES

Execuţia codului într-un proces

- Zone de cod
- Instruction pointer
  - poziţia curentă
  - incrementat cu dimensiunea unei instrucţiuni
- Fluxul se schimbă la branch-uri sau apeluri de funcţii
- Pointeri de funcţii

Bug-uri şi vulnerabilităţi

- Ce este un bug?
- Când este un bug o vulnerabilitate?
- Ce obiective sunt în exploatarea unei vulnerabilităţi?

Tipuri de vulnerabilităţi la nivelul memoriei

- Suprascrierea datelor cu date de atac
  - suprascriere variabile (verificate în if)
  - suprascriere pointeri de funcţii
- Alterarea fluxului de execuţie
- Executarea de apeluri arbitrare
- Executarea de cod arbitrar (code injection)

exec("/bin/bash")

- Sau echivalentul system("/bin/bash")
- Obţinerea unui shell într-o aplicaţie existentă
- Obiectivul iniţial al unui atac
- Atacatorul se va folosi de vulnerabilităţi ale memoriei pentru a porni un shell
- Ulterior:
  - obţinere informaţii confidenţiale
  - denial of service
  - privilege escalation

STACK BUFFER OVERFLOW

Injectare de cod

- Punere de cod într-o zonă writable şi executarea sa
- Să putem scrie cod
  - Folosim funcţii de citire input (fgets, scanf)
- Să putem executa cod de acolo
  - zonă executabilă
  - jump la acea adresă
- E nevoie de o zonă simultan writable + executable

Shellcode
• Instrucţiuni în cod binar
• Se scriu într-o zonă / buffer
• Se “sare” aici pentru execuţie
• De obicei realizează exec(“/bin/bash”)
• Se încearcă injectarea acestuia într-o zonă accesibilă

Structura stivei (reminder)
• Adresă de retur
• Fostul frame pointer
• Variabile locale (buffere incluse)
• Dacă un buffer este neîncăpător (overflow) putem suprascrie date, eventual chiar adresa de retur

Stack buffer overflow
• Scrierea în buffer peste dimensiunea alocată
• De ce merge?
  – avem spaţiu alocat pe stivă
• Ce suprascriem?
  – alte variabile
  – adresa de retur
• Ce obţinem
  – alterarea fluxului de execuţie
  – rulare de cod arbitrar

Suprascrierea adresei de retur
• Adresa pe care o va folosi apelatul la încheierea funcţiei
  – este adresa instrucţiunii următoare faţă de cea folosită de apelant în momentul apelului
• Se găseşte pe stivă
• Stack buffer overflow poate conduce la suprascrierea adresei de retur
• Se pune adresa shellcode-ului
• Shellcode-ul poate sta pe stivă

Funcţii de lucru cu şiruri
• str*
• Împreună cu funcţiile de citire de intrare (fgets) sunt principalele funcţii exploatabile
• NBTS (NUL -terminated byte strings)
  – dacă un şir nu e NUL-terminat, putem suprascrie dincolo de dimensiunea sa
• Lungimea unui şir
  – trebuie ştiută tot timpul
  – presupunei legate de lungimea şirului pot conduce la vulnerabilităţi de securitate

METODE OFENSIVE ŞI MECANISME DEFENSIVE

Injectare de cod

- Zone writable + executable
- Se poate plasa cod acolo şi executa
- Acest cod se numeşte “shellcode”
- Clasic: pe stivă + stack buffer overflow
  - citire buffer de la standard input
  - variabilă de mediu

DEP

- Data execution prevention
- O zonă writable nu este executable
- W^X
- NX flag pe arhitecturile moderne
- Bypass: apel mprotect, VirtualProtect

return-to-libc

- Executarea unei funcţii existente
- Clasic, system(“/bin/bash”)
- Nu este nevoie de injectare de cod
- Trebuie ştiută adresa funcţiei

Canary value

- Plasată pe stivă între variabile locale şi adresa de retur
- Se verifică coerenţa informaţiei la părăsirea funcţiei
- Previne efectul stack buffer overflows pentru a suprascrie adresa de retur

Suprascriere de alte date

- Nu suprascriem adresa de retur
- Variabile locale alterează fluxul de lucru
- Pointeri de funcţii
- Suprascriere pointeri de gestiune a heap -ului

ASLR

- Randomizează plasarea bibliotecilor
  - dificil de găsit adresa funcţiilor de bibliotecă
  - dificil de realizat return-to-libc
- Randomizează plasarea stivei
  - adresa de retur se suprascrie cu o adresă efectivă, nu cu o funcţie cu jump relativ
  - adresa buffer-ului de pe stivă variază la fiecare rulare
  - dificil de ştiut unde să sari pe stivă

CONCLUZII
Safe coding
• Atenţie la funcţii de lucru pe şiruri
• Folosiţi suport DEP, ASLR, canary value
• Analiză statică pentru verificarea de bug-uri sau vulnerabilităţi

Curs 8 - Fire de executie

Cuprins
- Server de web
- Thread-uri
- API
- Implementare
- Sincronizare

Cum implementam un server de web?
- Cerinte:
  - Serveste un numar arbitrar de clienti
  - Fiecare cilent poate cere oricate pagini (HTTP 1.1)
  - Mentine statistici: nr total de pagini vizionare, nr total de clienti, octeti cititi etc.

Implementare secventiala

```
while(1){
        int s = accept(ls);
        fname = read_request(s);
        while(fname){
                read_and_send_file(fname);
                update_stats();
                fname = read_request(s);
        }
}
```

Probleme
- Un singur client simultan
- Ineficient chiar si cu un singur procesor

ALTERNATIVE
Folosind procese

```
while(1){
        int s = accept(ls);
        if(for()==0){
                fname = read_request(s);
                while(fname){
                        read_and_send_file(fname);
                        update_stats();
                        fname = read_request(s);
                }
        }
        else {...}
}
```

Probleme

- Cum actualizam statisticile?
- Cost mare pentru pornire proces

Implementare asincrona

```
while(1){
        int s = accept(ls);
        add_client(s);
        select(...);
        for(c:clients){
                if(FD_ISSET(c.s)){
                        fname = read_request(s);
                        c.d = open(fname, ...);
                        c.status = read_file;
                        //….
                }else if (FD_ISSET(c.d)){
                        read(c.d, buf, 1000);
                        send(c.s, buf, 1000);
                        ...
                }
        }
}
```

Probleme

- Trebuie sa tinem stare pentru fiecare client
- Greu de implementat

Am dori o primitiva SO care:

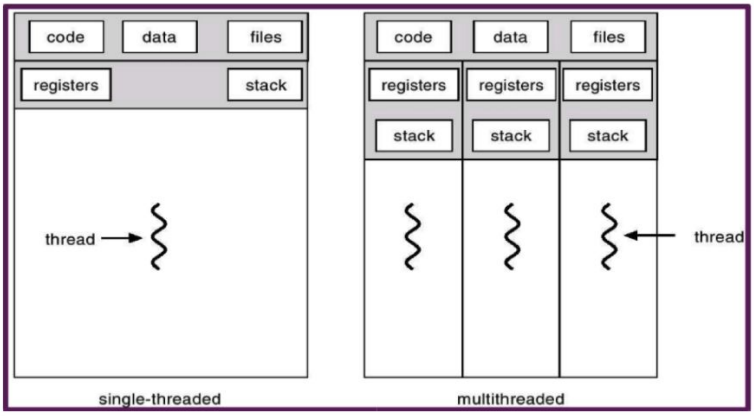
- Executa secvential un set de instructiuni
- Este usor de pornit/oprit
- Partajeaza date cu usurinta

Folosind thread-uri

```
while(1){
        int s = accept(ls);
        pthread_create(&t, NULL, (void*)&cnt, (void*)&s);
}
…
void* clnt(void* p){
        Int s = *(int*)p;
        Char fname = read_request(s);
        while(fname){
                read_and_send_file(fname);
                update_stats();
                fname = read_request(s);
        }
}
```

Fir de executie(thread)

- O secventa de control in cadrul unui proces
- Executa secvential un set de instructiuni
- Un proces are unul sau mai multe thread-uri care partajeaza resursele sale



Ce partajeaza thread-urile?

- Variabile globale (.data, .bss)
- Fisiere deschise
- Spatiul de adresa
- Modul de tratare a semnalelor

Ce nu partajeaza thread-urile?

- Registrele
- Stiva
- Program counter/ instruction pointer
- Stare
- Masca de semnale
- TLS (Thread Local Storage)

Procese vs thread-uri

Procese	Thread-uri
Grupeaza resurse	Abstractizeaza executia
Fisiere, lucru retea	Stiva
Spatiu adrese	Registri
Fire de executie	Program counter

Avantaje thread-uri

- Timp de creare mai mic decat al proceselor
- Timp mai mic de schimbare de context
- Partajare facila de informatie
- Utile chiar si pe uniprocesor

Dezavantaje thread-uri

- Daca moare un thread, moare tot procesul
- Nu exista protectie la partajarea datelor
- Probleme de sincronizare
- Prea multe thread-uri afecteaza performanta!

Operatii cu thread-uri

- Lansarea in executie
- Incetarea executiei
- Terminare fortata (cancel)
- Asteptare (join)
- Planificare

Posix Threads

- Folosit pe sistemele Unix
- API pentru crearea si sincronizarea thread-urilor
- Folosire

- Inclus header-ul (#incluse <p.thread.h>)
- Legarea bibliotecii (-lpthread)
- Man 7 pthreads

API PThreads
pthread_t tid;
pthread_create(&tid, NULL, threadfunc, (void*)arg);
pthread_exit(void* ret);
pthread_join(pthread_t tid, void** ret);
pthread_cancel(pthread_t tid);

Thread-uri in Linux
- Suport kernel pentru task-uri (struct task_struct)
- Procesele si thread-urile sunt task-uri
- Planificabile independent
- NPTL (New Posix Thread Library)
    - Implementare pthreads(1:1)
    - Foloseste apelul de sistem clone
    - Thread-urile sunt grupate in acelasi grup
    - Getpid intoarce thread group ID

Clone
- Specific Linux
- Folosit de fork si NPTL
- Diferite flag-uri specifica resursele partajate
    - CLONE_NEWNS
    - CLONE_FS, CLONE_VM, CLONE_FILES
    - CLONE_SIGHAND, CLONE_THREAD

Thread-uri in Windows
- Model hibrid: suport in kernel
- Fibre: fire de executie in user-mode
    - Planificate cooperativ
    - Blocarea unei fibre blocheaza firul de executie

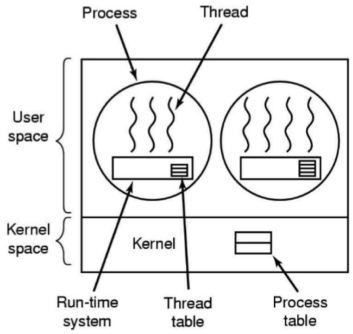
API Windows
HANDLE CreateThread(...)
ExitThread
WaitForSingleObject/ MultipleObjects
GetExitCodeThread
TerminateThread
TlsAlloc
TlsGetValue/TlsSetValue

IMPLEMENTARE THREAD-URI

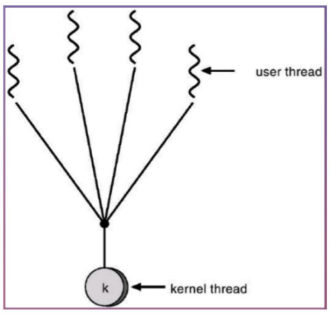
User-level

- O biblioteca de thread-uri ofera suport pentru crearea, planificarea si terminarea thread-urilor
- Mentine o tabela cu fire de executie: PC, registre, stare pentru fiecare fir
- Nucleul “vede” doar procese, nu si thread-uri
- Mai multe fire de executie sunt planificate cu un singur proces

Thread-uri implementate user-level



Mai multe fire de executie sunt mapate pe acelasi fir de executie din kernel



Avantaje

- Usor de integrat in SO:nu sunt necesare modificari
- Pot oferi suport multithreaded pe un SO fara suport multithreaded
- Schimbare de context rapida: nu se executa apeluri de sistem in nucleu
- Aplicatiile pot implementa planificatoare in functie de necesitati

Dezavantaje

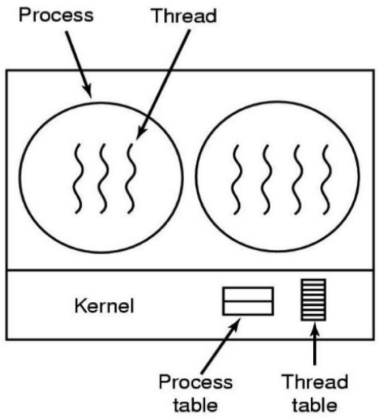
- Un apel de sistem blocant blocheaza intrg procesul
- Un page-fault blocheaza tot procesul
- Planificare cooperativa
- Multe aplicatii folosesc apeluri de sistem oricum

Kernel level threads

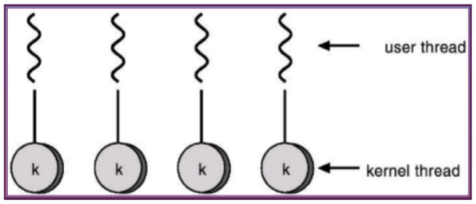
- Suport in kernel pentru creare, terminare si planificare
- Model unu-la-unu

Avantaje	Dezavantaje
Fara probleme la apeluri blocante sau page faults	Crearae si schimbarea de context e mai lenta
Pot fi planificate pe sisteme multiprocesor	

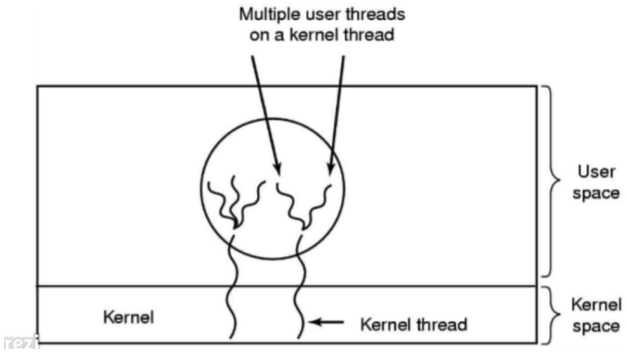
Implementare thread-uri in kernel



Un kernel thread pentru fiecare thread utilizator



Hibrid



Sincronizare

- Thread-urile ofera acces la date comune
- Accesul trebuie mediat pentru a implementa programe corecte
- Chiar si programele cu un singur thread pot crea probleme
- Dar cele cu mai multe thread-uri?

Exemplu

```
int total_bytes;
void update_statics(int j){
        total_bytes+=j;
}
Void signral_handler(){
        update_statistics(1);
}
```

Reentranta

- O functie este reentranta daca poate fi executata simultan de mai multe ori, fara a afecta rezultatul
- Conditii necesare:
    - Nu lucreaza cu variabile globale/statice
    - Apeleaza doar functii reentrante
- Reentranta este importanta(mai ales) in programe cu un singur thread din cauza semnalelor

Reentranta in practica

- Multe functii de biblioteca seteaza variabila errno
- Depinde de implementare daca acestea sunt reentrante:
    - Anumite apeluri au veriuni reentrante: gethostbyname_r
    - Activare cu macroul _REENTRANT

Thread safety

- O functie este thread-safe daca poate fi apelata din mai multe thread-uri in acelasi timp
- Strategii implementare
    - Acces exclusiv
    - Semafoare
    - Monitoare
    - Thread-local storage
    - Reentranta
    - Operatii atomice

Curs 9 - Sincronizare

Cuprins
• Recapitulare thread-uri
• Nevoia de sincronizare
• Sincronizarea la procese şi thread-uri
• Moduri de sincronizare
• Implementarea sincronizării
• Problematica sincronizării

NEVOIA DE SINCRONIZARE

Nevoia de sincronizare

```
unsigned long sum = 0;
void thread_func(size_t i)
{
        sum += i * i * i;
        print("sum3(%zu): %lu\n", i, sum);
}
int main(void) {
         size_t i;
        for (i = 0; i < NUM_THREADS; i++)
                create_thread(thread_func, i);
        [...]
}
```

• Acces exclusiv / serializare / atomizare
  – Regiune critică
• Secvenţiere / ordonare
  – Read after write, write after write, use after create
• Nedeterminism
  – Poate să meargă adesea deşi sunt probleme

Interacţiune în sistemele de calcul

• Între componente hardware
• Între procese/thread-uri
• Între user space şi kernel space
• Între aplicaţii şi hardware
• Metode
  – Notificări
  – Transfer de date (e.g. message passing)
  – Date partajate (e.g. shared memory)

Exemple

1. Placa de reţea primeşte pachete pe care să le transfere procesorul sau DMA-ul în memorie
2. Două procesoare au nevoie de acces la aceleaşi date din memoria RAM.
3. Un proces solicită sistemului de operare aducerea de date de pe disc.
4. Un proces aşteaptă încheierea unei acţiuni a altui proces.
5. Mai multe thread-uri execută operaţii pe o structură de date comună (partajată).

Probleme

1. Placa de reţea primeşte mai multe informaţii decât poate procesorul prelucra; se pierd informaţii.
2. Ambele procesoare încearcă să modifice aceeaşi dată. Data este acum incoerentă/coruptă.
3. Bufferul de memorie unde sistemul de operare stochează datele este suprascris cu alte date (cerute de alt proces).
4. Procesul “pierde” notificarea din partea celuilalt proces şi aşteaptă nedefinit.
5. Thread-urile modifică în mod concurent structurile de date (vectori, liste, matrice, întregi, pointeri) şi acum sunt incoerente/corupte.

Producător-consumator cu probleme

producer

```
wait(buffer_not_full);
produce_item();
signal(buffer_not_empty);
```

consumer

```
wait(buffer_not_empty);
consume_item();
signal(buffer_not_full);
```

Producător-consumator corect

producer

```
lock(mutex);
if (is_buffer_full())
   wait(buffer_not_full,mutex);
produce_item();
signal(buffer_not_empty);
unlock(mutex);
```

consumer

```
lock(mutex);
if (is_buffer_empty())
   wait(buffer_not_empty, mutex);
consume_item();
signal(buffer_not_full);
unlock(mutex);
```

Cod pe un server web

```
def add_quest_points(quest_level):
        points = get_current_player_points()
        points += points_for_level(quest_level)
        save_player_points(points)
        increase_player_quest_level()
def add_challenge_points(points):
        ... /* something similar to above */
```

Scenarii la codul de pe serverul web

• Se apelează simultan funcţia add_quest_points şi funcţia add_challenge_points

– Jucătorul rezolvă quest-ul şi altceva generează apelul pentru challenge

– Poate există quest-uri pe echipă şi simultan doi jucători rezolvă părţi din acelaşi quest (sau quest -uri diferite)

• Jucătorul apasă (conştient sau nu) de mai multe ori în browser pe butonul “Submit quest answer” (ajung două cereri identice la server). Care va fi noul nivel în quest al jucătorului? Ce punctaj va avea?

• Serverul web poate fi multi-proces sau multi-thread. Sincronizarea se face diferit în acele cazuri.

Sincronizare

• Pot apărea probleme din interacţiune

– Pierdere notificări, pierdere date

– Date corupte, date incoerente

• Sincronizarea asigură

– Comportament determinist

– Date coerente

• Moduri

– Secvenţiere/ordonare operaţii (A după B)

– Acces exclusiv la resurse (doar A sau doar B)

Sincronizarea prin secvenţiere

• O acţiune înaintea alte acţiuni

• Exemplu: operaţia wait/waitpid

• Este nevoie ca un proces/thread să execute o acţiune şi să producă un efect care să declanşeze acţiunea unui alt proces

• Exemplu: Thread-ul boss trebuie să producă “work item-ul” şi să îl adauge în coadă înainte ca un thread worker să îl preia

• În absenţa sincronizării, se vor prelua date nevalide şi programul va avea comportament arbitrar

Sincronizarea prin acces exclusiv

• Două sau mai multe procese/thread-uri au nevoie la aceleaşi date

• Exemplu: Mai multe thread-uri lucrează pe o matrice partajată

• Accesul exclusiv permite unuia singur accesul

– Serializare

– Regiune critică

• Dacă au nevoie toate să citească nu e nevoie de sincronizare

• Dacă unul scrie e nevoie de sincronizare prin acces exclusiv

– E posibil ca atunci când unul scrie altul să citească/parcurgă (de exemplu, într-o listă înlănţuită)

• În absenţa accesului exclusiv, cititorul va folosi structuri în stare incoerentă sau corupte

SINCRONIZAREA LA PROCESE ŞI THREAD-URI

Când folosim thread-uri

- Multi-core programming
  - CPU intensive: libx264 (encoding), comprimare, criptare
- În medii ce indică folosirea thread -urilor
  - Java
  - kernel development
  - OpenMP
- Pentru HPC (High Performance Computing)

Când nu folosim thread-uri

- Pe sisteme single core (dar există cazuri de utilizare şi acolo)
- Atunci când nu suntem confortabili cu API -ul
- Atunci când avem programe seriale (nu paralelizăm ce nu este nevoie)
- Atunci când ţinem foarte mult la robusteţea programului

Sincronizarea proceselor

- În general implicită
  - socketi
  - message passing, message queues
- Explicită
  - mai rar primitive de sincronizare “clasice” (mutexuri, semafoare, file locks)
  - memorie partajată (acces exclusiv)
  - semnale (secvenţiere)
  - API de lucru cu procese: wait

Sincronizarea thread-urilor

- În general explicită
- Date partajate, concurenţă: acces exclusiv
  - operaţii atomice
  - mutex -uri
  - spinlock-uri
- Operaţii secvenţiale: ordonare operaţii
  - semafoare
  - variabile condiţie
  - bariere

MODURI DE SINCRONIZARE

Date coerente şi date corupte

- Datele coerente “au sens”, determină comportament determinist pentru aplicaţie
- Interacţiunea necorespunzătoare (nesincronizată) produce date incoerente
- Exemplu: read before write

• Dacă un pointer a fost citit dar neiniţializat ...
• Nevoie de sincronizare

Sincronizare implicită
• Primitivele folosite nu au ca obiectiv sincronizarea
– Sincronizarea este un efect secundar (implicită)
• Transfer de date/mesaje: sockeţi, pipe-uri, cozi de mesaje, MPI
• Date separate/partiţionate
• Avantajos: uşor de folosit de programator

Transferul datelor
• Metode: read/write
– read: în general blocant, aşteaptă date scrise
– write: poate fi blocant dacă e bufferul plin
• Sincronizare implicită
– nu există date comune, datele sunt copiate/duplicate
– secvenţiere, ordonare: read-after-write
• E nevoie de sincronizare explicită pentru mai mulţi transmiţători sau receptori
• Avantaje: sincronizare implicită, uşor de programat, sisteme distribuite
• Dezavantaje: overhead de comunicare, blocarea operaţiilor

Partiţionarea datelor
• Se împart datele de prelucrat
• Fiecare entitate lucrează pe date proprii
• În mod ideal nu există comunicare între entităţi
– realist, e nevoie de comunicare (cel puţin agregarea datelor finale)
• Avantaje: reducerea zonelor de sincronizare, programare facilă
• Dezavantaje: date duplicate, posibil overhead de partiţionare şi agregare, aplicabilitate redusă

Sincronizare explicită
• Referită doar ca “sincronizare”
• Metode/mecanisme specifice de sincronizare
• Două forme de sincronizare
– pentru date partajate: primitive de acces exclusiv
– pentru operaţii secvenţiale: primitive de secvenţiere, ordonare
• Folosită atunci când este nevoie
• Menţine coerenţa datelor

Acces exclusiv
• În cazul datelor partajate
• Thread-urile/procesele concurează la accesul la date
– thread-urile modifică simultan date -> date incoerente – race condition (condiţie de cursă)

• Acces exclusiv
    – delimitarea unei zone în care un singur thread are acces
    – regiune critică (critical section)
    – e vorba de date, nu cod
• Metode: lock şi unlock, acquire şi release
• Primitive: operaţii atomice, mutex-uri, spinlock-uri
• Avantaje: date coerente
• Dezavantaje: overhead de aşteptare, cod serializat

Atomicitate
• Accesul la date să fie realizat de un singur thread
• Accesul exclusiv se referă la metodă
• Atomicitatea este o condiţie a zonei/datei protejate
• Datele/zonele neatomice partajate sunt susceptibile la condiţii de cursă
• Atomicitate: variabile atomice, primitive de acces exclusiv

Secvenţierea operaţiilor
• În cazul în care operaţiile trebuie ordonate
• Un thread scrie, un alt thread citeşte
    – e nevoie ca un thread să aştepte după altul
• Un thread produce, alt thread consumă
• Secvenţiere
    – un thread B se blochează în aşteptarea unui eveniment produs de un thread A
    – thread-ul B execută acţiunea după thread-ul A
• Metode: signal/notify şi wait
• Primitive: semafoare, variabile condiţie, bariere
• Avantaje: comportament determinist
• Dezavantaje: overhead de aşteptare

Folosire acces exclusiv
• Oricând avem date partajate
• Garantăm că un singur thread poate avea acces la acele date
• Codul este serializat (probleme de paralelism)
• Între lock şi unlock definim o regiune critică (serială)
• Pe cât posibil folosim operaţii atomice (rapide)
• Dacă regiunea critică este mică folosim spinlock-uri
• Dacă regiunea critică este mai mare folosim mutexuri

Folosire secvenţiere
• În general în probleme care se reduc la producătorconsumator
    – Un thread “produce” informaţii/date/structuri/pachete, iar altul “consumă”
• De obicei există un obiect de sincronizare şi o variabilă (flag) care indică producerea informaţiei

• Producătorul produce, pune flag-ul pe activ şi notifică obiectul de sincronizare
• Consumatorul verifică flag-ul, dacă e inactiv aşteaptă la obiectul de sincronizare până când este notificat şi apoi consumă informaţia

IMPLEMENTAREA SINCRONIZĂRII
Cadrul pentru sincronizare
• Mai multe entităţi active (procese/thread-uri)
• Puncte de interacţiune (date comune, evenimente)
• Preemptivitate
– O entitate activă poate fi întreruptă oricând de o altă entitate activă
• oricând = atunci când vine o întrerupere de ceas, se invocă planificatorul care poate decide trecerea entităţii active curente din RUNNING în READY şi planificarea alteia

Implementare simplă
• Un singur proces/thread (doar pentru medii specializate)
• Date complet partiţionate (izolare)
• Dezactivarea preemptivităţii
– configurare planificator
– dezactivarea întreruperilor (întreruperii de ceas)
– doar pentru sisteme uniprocesor
• Folosirea de operaţii atomice

Implementare mutex
• Un boolean pentru starea internă (locked/unlocked)
• O coadă cu thread-urile care aşteaptă eliberarea mutexului

```
struct mutex {
    bool state;
    queue_t *queue;
};
```

```
void lock(struct mutex *m)
{
    while (m->state == LOCKED) {
        add_to_queue(m->queue);
        wait();
    }
    m->state = LOCKED;
}
```

```
void unlock(struct mutex *m)
{
    m->state = UNLOCKED;
    t = remove_from_queue(m->queue);
    wake_up(t);
}
```


Nevoia de spinlock
• Funcţiile lock şi unlock pe mutex trebuie să fie atomice
• Dezactivarea preemptivităţii
• Folosirea unui spinlock
• Spinlock-ul este o variabilă simplă cu acces atomic
• Operaţiile folosesc busy waiting

• Pentru atomicitate: suport hardware
  – TSL (test and set lock)
  – cmpxchg (compare and exchange)

Implementare spinlock
• Un boolean/întreg cu acces atomic
• Atomic compare and exchange pentru lock

```
int atomic_cmpxchg(int val, int compare, int replace)
{   /* This is an equivalent implementation. */
    /* The entire operation is atomic. */
    int init = val;
    if (val == compare)
        val = replace;
    return init;
}
```

```
0 - locked
1 - unlocked
```

```
void lock(struct spinlock *s)
{
    while (atomic_cmpxchg(s->val, 1, 0) == 0)
        ;  /* do nothing */
}
```

```
void unlock(struct spinlock *s)
{
    atomic_set(s->val, 1);
}
```

Spinlock vs. mutex

Spinlock	Mutex
• Busy-waiting • Simplu • Pentru regiuni critice scurte	• Blocant • Coadă de așteptare • Pentru regiuni critice scurte • Pentru regiuni critice mari sau în care thread-ul se blochează

PROBLEMATICA SINCRONIZĂRII
Dacă nu folosim sincronizare ...
• Race conditions
  – read-before-write
  – write-after-write
  – Time Of Check To Time Of Use
• Date corupte, incoerente
• Comportament nedeterminist, arbitrar
• Procesul/thread-ul poate provoca un crash (acces nevalid la memorie)

Dacă folosim sincronizare ...
• Implementare incorectă
  – deadlock

  – aşteptare nedefinită (pierderea notificării)
• Implementare ineficientă
  – granularitate regiune critică
  – cod serial
  – thundering heard
  – lock contention
• Probleme implicite
  – overhead de aşteptare
  – overhead de apel

Deadlock
• Deadly embrace
• Două sau mai multe thread-uri aşteaptă simultan unul după altul
  – A obţine lock-ul L1 şi aşteaptă după lock-ul L2
  – B obţine lock-ul L2 şi aşteaptă după lock-ul L1
• Lock-urile trebuie luate în ordine ca să prevină deadlock
• Forma de deadlock cu spinlock-uri: livelock

Aşteptare nedefinită
• A aşteaptă un eveniment
• B nu produce evenimentul (sau A pierde notificarea)
• A aşteaptă nedefinit (evenimentul nu se produce)

Granularitatea regiunii critice
• Regiune critică mare
  – mult cod serial, paralelism redus
  – legea lui Amdahl
• Regiune critică mică
  – overhead semnificativ de lock şi unlock faţă de lucrul efectiv în zonă
  – lock contention (încărcare pe lock)

Thundering herd
• În momentul apelului unlock() sau notify() sunt trezite toate thread-urile
• Toate thread-urile încearcă achiziţionarea lock-ului
• Doar unul reuşeşte, restul se blochează
• Operaţia este reluată la următorul apel unlock() sau notify()

Overhead de apel
• Apelurile de lock, unlock, wait, notify sunt costisitoare
• În general înseamnă apel de sistem
• De multe ori invocă planificatorul
• Mai multe apeluri, mai mult overhead
• Lock contention generează mai mult overhead

• De preferat, unde se poate, operaţii atomice

CONCLUZII
Când folosim sincronizare explicită
• Când e neapărat nevoie
• În cazul în care avem date comune (partajate)
• În cazul în care avem nevoie de secvenţiere de operaţii
• Nu am putut separa/partiţiona datele • Nu am putut folosi transfer de date/mesaje (sincronizare implicită)
• Sincronizarea este un rău necesar
– rău: overhead, regiuni seriale, deadlock la folosire necorespunzătoare
– necesar: fără sincronizare obţinem date corupte, comportament arbitrar, crash-uri, vulnerabilităţi de securitate

Dacă folosim sincronizare explicită ...
• Ne referim la date (nu la cod)
• Gândim bine soluţia
• Avem grijă la dimensiunea regiunilor critice
• Folosim variabile atomice unde se poate
• Pentru regiuni critice mici folosim spinlock-uri
• Pentru regiuni critice mari folosim mutexuri
• Nu folosim mai multe thread-uri decât avem nevoie
– pool de thread-uri
– modelul boss/workers
– modelul worker threads

Curs 10 - Dispozitive de intrare/iesire

Dispozitive de intrare/iesire
Int fd = open/scket/pipe…;
read(fd, buf, 10);
write(fd2, buf, 10);
close(fd);

Dispozitivele de I/O sunt foarte diverse

aspect	variation	example
data-transfer mode	character block	terminal disk
access method	sequential random	modem CD-ROM
transfer schedule	synchronous asynchronous	tape keyboard
sharing	dedicated sharable	tape keyboard
device speed	latency seek time transfer rate delay between operations	
I/O direction	read only write only read–write	CD-ROM graphics controller disk

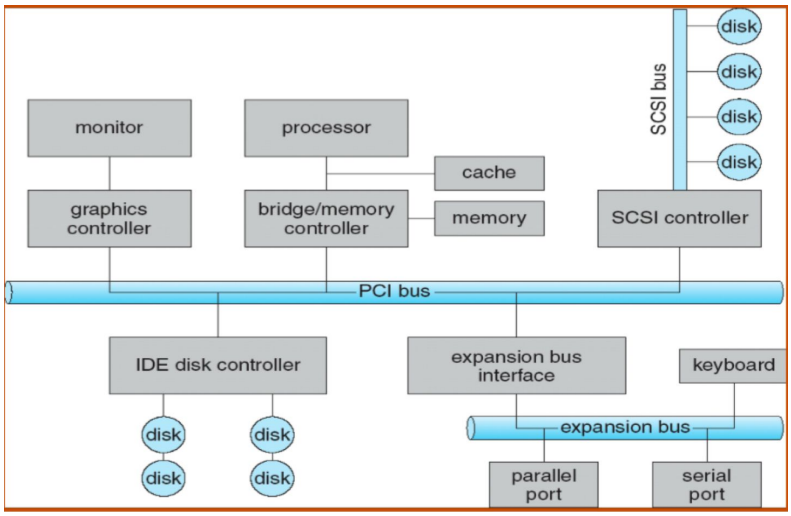
Ce vor utilizatorii?

- Usurinta in utilizare:
    - Cod care ruleaza pe oricate dispozitive de intrare-iesire
    - Sa fie usor sa numeasca dispozitivele
    - Tratarea erorilor sa se intample automat
- Performanta
- Operatii dorite:
    - Sincrone (majoritatea)
    - asincrone(unele aplicatii)

CUM IMPLEMENTAM?
Hardware

- Doua categorii de dispozitive: caracter si bloc
- Fiecare dispozitiv are
    - Un controller
    - Unul sau mai multe porturi
    - Conectat la o magistrala
- Procesorul da comenzi I/O controllerului
- Controllerul gestioneaza dispozitivul
- 2 categorii de dispozitive: caracter si bloc
- Fiecare dispozitiv are:

    - Un controller
    - Unul sau mai multe porturi
    - Conectat la o magistrala
- Procesorul da comenzi de I/O controllerului
- Controllerul gestioneaza dispozitivul



Comunicatia cu dispozitivele de I/E

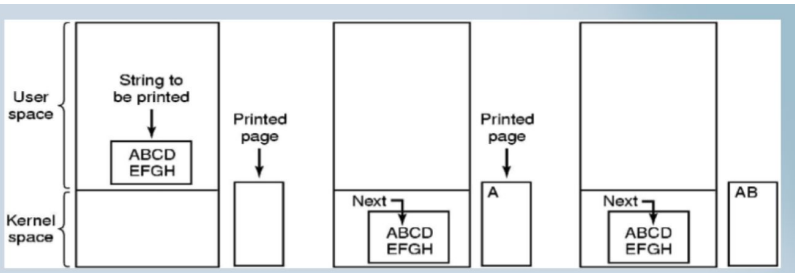
- Fiecare controller are:
    - Registre de stare, control, intrare, iersire
    - (optional) buffer pentru date

Port mapped I/O

- Spatiu de adresaseparat pentru dispozitivele de I/O
- O adresa de I/O se numeste port
- Un port adreseaza un registru al controllerului
- Instructiuni specializate:
    - IN REG, 0x2F8
    - OUT REG, 0x2FA

Memory mapped I/O

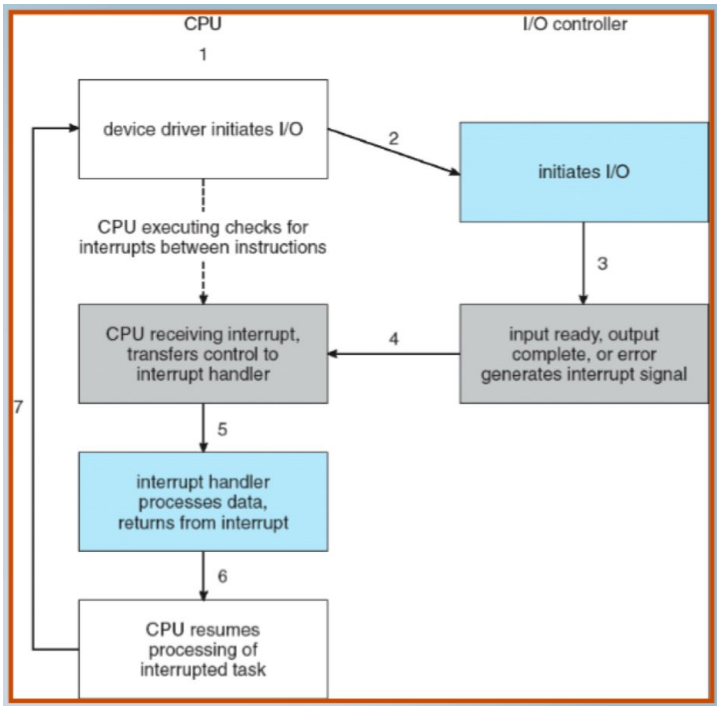
- Registrele I/O sunt mapate in spatiul de memorie
- Avantaje:
    - Nu necesita instructiuni specializate
    - Protectia de la memoria virtuala
    - Instructiunile pot referi memorie sau registre
- Dezavantaje:
    - Trebuie inhibat cache-ul la nivel de pagina
    - Bridge-uri intre magistrale -> mai lent decat port-mapped I/O




```
copy_from_user(buffer, p, count);
for (i=0;i<count;i++){
        while (*printer_status_req!=READY);
       printer_data_register = p[i];
}
return_to_user();
```

Intreruperi

- Anunta procesorul ca s-a intamplat un eveniment care trebuie tratat
- Pot fi mascabile sau nemascabile
- Sunt generate de software sau hardware (controller)
- Folosesc linii de intrerupere (IRQ line)
- Exista o tabel cu rutine de tratare a intreruperilor
- Procesorul ruleaza rutina de tratare a intreruperii (ISR)



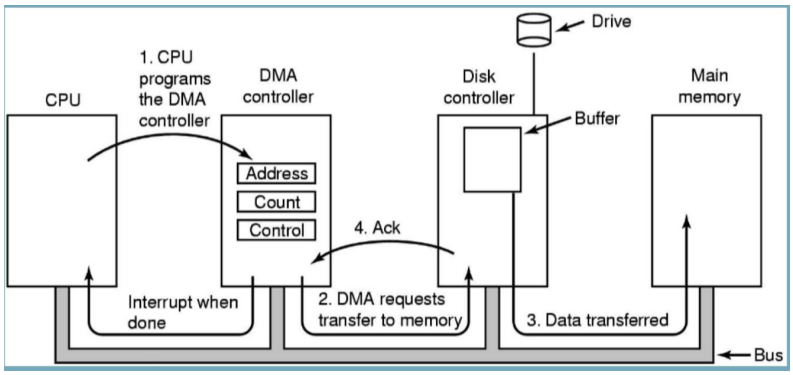
Implementare cu intreruperi

```
copy_from_user(buffer, p , count);
enable_interrupts();
while (*printer_status_reg!=READY);
*printer_data_register = p[0];
scheduler();
```

```
if (count==0) {
    unblock_user();
}
else {
    *printer_data_register = p[i];
    count --; i++;
}
acknowledge_interrupt();
return_from_interrupt;
```

Direct Memory Access

- DMA: dispozitiv separat specializat pentru copieri din memorie in memorie
- CPU comanda DMA sa efectueze transferuri date ca (sursa, destinatie, octeti)
- CPU nu poate accesa magistrala daca este folosita de DMA

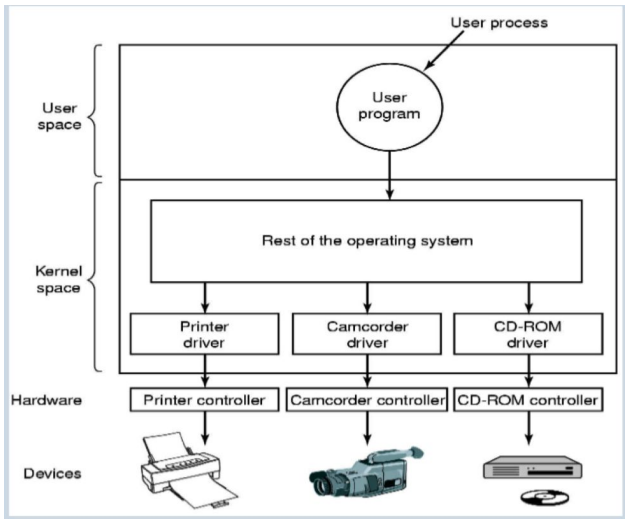


Implementare cu DMA

```
copy_from_user(buffer,p,count);
set_up_DMA_controller();
scheduler();
```

```
acknowledge_interrupt();
unblock_user();
return_from_interrupt();
```

Device Drivers

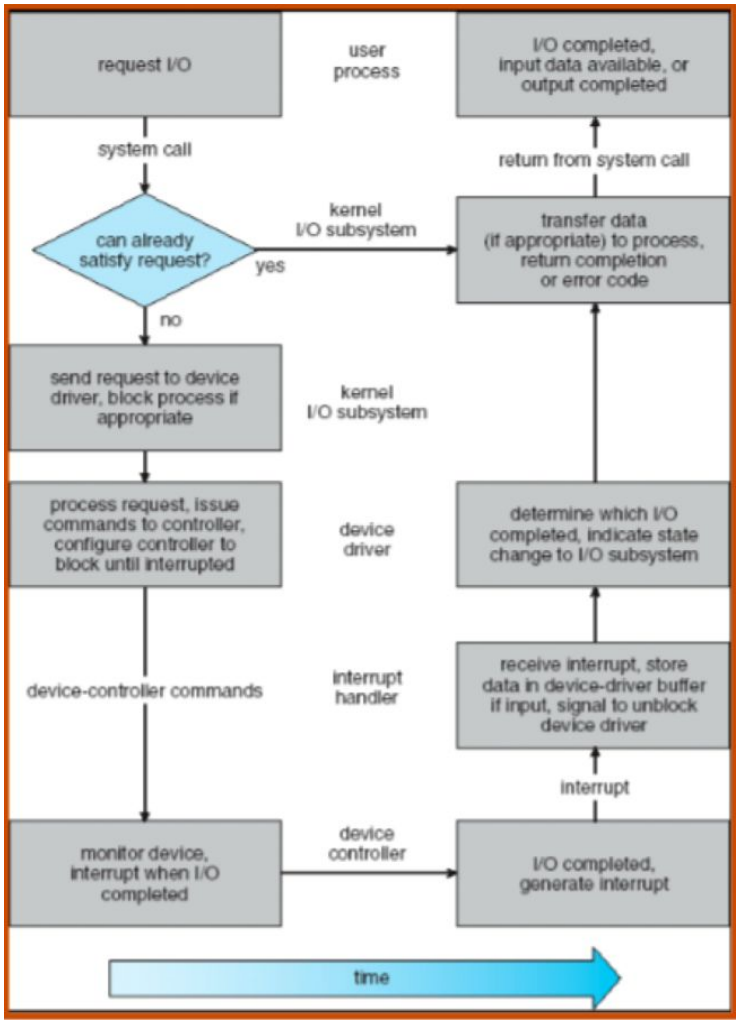


- Controleaza un dispozitiv sau clasa de dispozitive
- Ruleaza in kernel
- SO ofera interfata comuna pentru drivere

Operatii independente de dispozitiv

- Planificare pentru eficienta si echitate
    - Sortare si comasare de cereri de I/O
- Buffering
    - Compensarea vitezei de transfer dintre dispozitive
    - Reasamblarea datelor
    - Semantica de copiere pentru utilizatori
- Caching: memoria tine copii ale datelor
- Spooling: se mentine un buffer cu datele transmise unui dispozitiv
- Rezervarea dispozitivului

Fluxul unei operatii de I/O



API Utilizator

Char Devices	Block Devices	Network
Ex: mouse, tastatura	Ex: discuri, CDROM	Ex: 802.11, 802.5
Acces secvential	Acces aleator	Separare protocol de interfata
Operatii: put, get	Operatii: read, write, seek	sockets
Transfer la nivel de caracter	Transfer la nivel de bloc	Operatii: connect, send, recv, close
Viteza redusa	Viteza ridicata	Viteza ridicata

Initializare

- Dispozitive block/caracter: opern, close
- Dispozitive retea: socket, connect, shutdown

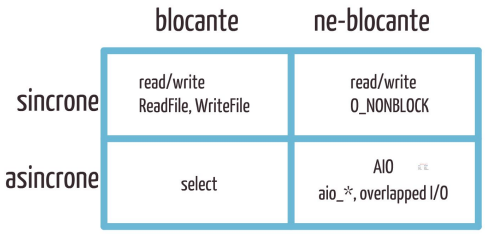
Control

- Controleaza dispozitivul de I/O
- Depinde de dispozitiv!
  - ioctl/ DeviceIOControl
  - Setsockopt, getsockopt

Tipuri de operatii I/O

Blocante	Ne-blocante	Asincrone
- Procesul este suspendat pana la incheierea operatiei - Simplu de folosit	- Operatia se intoarce imediat - Procesul primeste datele disponibile	- Procesul ruleaza in paralel cu operatia - Este notificat atunci cand operatia este finalizata - Event-driven programming

Tipuri de operatii I/O



Asynchronous I/o

- Unix: aio_read, aio_write, aio_suspend
- Windows: ReadFile, WriteFile, OVERLAPPED I/O